JSPM's

RAJARSHI SHAHU COLLEGE OF ENGINEERING

TATHAWADE, PUNE-33

(An Autonomous Institute Affiliated to Savitribai Phule Pune University,Pune)

LAB MANUAL OF

Deep Learning (IT-4102)

B.Tech(2025-26) INFORMATION TECHNOLOGY

Version: - 3

© JSPM Group of Institutes, Pune. All Rights Reserved. All the information in this Course Manual is confidential. Participants shall refrain from copying, distributing, misusing/ or disclosing the content to any third parties any circumstances whatsoever.

INDEX			
Sr. No.		**Topic**	**Page. No.**
1		**How to Use This Manual…….**	**4**
2		**Laboratory Details……**	**5**
	2.1	Laboratory Objective	**9**
	2.2	Laboratory Equipment	**9**
	2.4	Experiment No.1	**11**
		Practical Session Plan	**11**
		Aim, theory, procedure, observations, calculations etc	**12-21**
	2.5	Experiment No.2.	**22**
		Practical Session Plan	**22**
		Aim, theory, procedure, observations, calculations etc	**23-29**
	2.6	Experiment No.3	**30**
		Practical Session Plan	**30**
		Aim, theory, procedure, observations, calculations etc	**31-39**
	2.7	Experiment No.4	**40**
		Practical Session Plan	**40**
		Aim, theory, procedure, observations, calculations etc	**41-46**
	2.8	Experiment No.5	**47**
		Practical Session Plan	**47**
		Aim, theory, procedure, observations, calculations etc	**48-53**
	2.9	Experiment No.6	**54**
		Practical Session Plan	**54**
		Aim, theory, procedure, observations, calculations etc	**55-59**

PSCOE-IT

1. How to Use This Manual

This manual assumes that the facilitators are aware of collaborative learning methodologies.

This manual will provide a tool to facilitate the session on Deep Learning Subject in collaborative learning environment.

The facilitator is expected to refer this manual before the session.

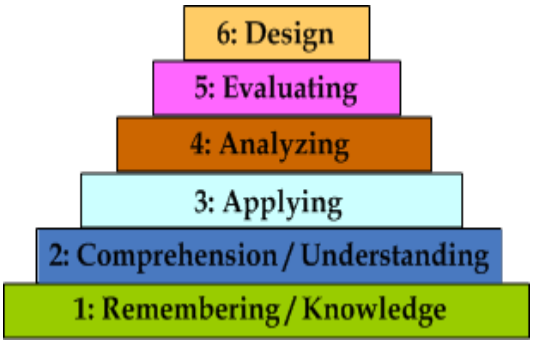
Icon of Graduate Attributes

K Applying Knowledge	**A** Problem Analysis	**D** Design & Development	**I** Investigation of problems
M Modern Tool Usage	**E** Engineer & Society	**E** Environment Sustainability	**T** Ethics
T Individual & Team work	**O** Communication	**M** Project Management & Finance	**I** Life Long Learning

Disk Approach- Digital Blooms Taxonomy



1: Remembering / Knowledge
2: Comprehension / Understanding
3: Applying
4: Analyzing
5: Evaluating
6: Creating / Design



This Manual uses icons as visual cues to the interactivities during the session.

	Blooms Taxonomy
	Remembering
	Understanding
	Applying
	Analyzing
	Evaluating
	Creating

	This icon is used to indicate instructions for faculties.
	This icon is used to indicate a statement to be made by faculty.
	This icon is used to indicate a list of additional resources.
	This icon indicates an activity to be conducted.
	This icon indicates questions to be asked by faculty.

RSCOE-IT

VISION of Institute

To satisfy the aspirations of youth force, who want to lead the nation towards prosperity through techno economic development.

MISSION of Institute

To provide, nurture and maintain an environment of high academic excellence, research and entrepreneurship for all aspiring students, which will prepare them to face global challenges maintaining high ethical and moral standards.

VISION of Department

To create quality information technology professionals through superior academic environment.

MISSION of Department

To incorporate the IT fundamentals in students to be successful in their career. To motivate the students for higher studies, research and entrepreneurship. To provide IT services to society.

PSO's of Department

PSO1: Demonstrate the ability to apply discrete principles of mathematics along with programming paradigms to expedite solution building in the IT domain.

PSO2: Apply computational techniques using core aspects of network and system programming to deliver secured applications in the arena of analytics and computing.

PSO3: Demonstrate project management skills to handle multidisciplinary complex tasks proficiently and utilize these skills for entrepreneurship.

RSCOE-IT

PROGRAM OUTCOMES (POs)

Engineering Graduates will be able to:

PO 1: Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO 2. Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO 3. Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO 4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO 5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO 6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO 7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO 8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO 9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO 10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO 11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO 12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

RSCOE-IT

Course Prerequisites:

Engineering Drawing and Computer Aided Graphics, Basic Geometry, Trigonometry, Vectors and Matrices, Basics of Data Structures and Algorithms.

Course Objectives:

- To present the mathematical, statistical and computational challenges of building neural networks
- To study the concepts of deep learning
- To learn different deep learning architectures
- To enable the students to know deep learning techniques to support real-time applications
- To examine the case studies of deep learning techniques.

Course Outcomes

CO1: Understand the fundamental concepts of deep learning, neural networks, and Implement basic neural networks, including perceptrons, multi-layer perceptrons (MLPs), and backpropagation algorithms.

CO2: Apply optimization techniques and dimensionality reduction methods like PCA and SVD to improve deep learning models.

CO3: Analyze the role of autoencoders, denoising autoencoders, and sparse autoencoders in unsupervised learning, and apply regularization techniques to improve model performance.

CO4 : Design and develop deep learning models using Convolutional Neural Networks (CNNs) and explore different architectures such as LeNet, AlexNet, VGG16, Inception 3, and EfficientNet B0.

CO5: Implement sequential models using Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM), Gated Recurrent Units (GRUs), and Bidirectional LSTMs for time-series and NLP applications.

CO6: Apply deep learning techniques in real-world scenarios, including ImageNet classification, WaveNet for speech processing, Word2Vec for NLP, bioinformatics, and face recognition.

RSCOE-IT

2.1 Laboratory Objective

Upon completion of Deep Learning Lab student should able to:

1. Classification with Multilayer Perceptron using Scikit-learn
2. Classification of MNIST Dataset using CNN
3. Face recognition using CNN
4. Time Series Prediction using RNN
5. Image generation using GAN
6. Sentiment analysis of text using LSTM

2.1 Laboratory Equipment list

Hardware requirements

Hardware	Specification
Processor	Intel Pentium 4 or higher, AMD Athlon 64, AMD Opteron processor, AMD Phenom processor
Video card	Qualified hardware-accelerated OpenGL graphics cards
Memory	2 GB, 4 GB for 64-bit OS
Hard drive	10 GB
Optical drive	DVD-ROM
Internet browser	Microsoft Internet Explorer 7.0 or higher, Apple Safari, or Mozilla Firefox, or Google Chrome

2.2 Laboratory Plan

Exp. No.	Title of Experiment
1.	Classification with Multilayer Perceptron using Scikit-learn (MNIST Dataset)
2.	Classification of MNIST Dataset using CNN
3.	Face recognition using CNN
4.	Time Series Prediction using RNN
5.	Image generation using GAN
6.	Sentiment analysis of text using LSTM
	Mock Practical Examination

RSCOE-IT

EXPERIMENT NO.1

Classification with Multilayer Perceptron using Scikit-learn (MNIST Dataset)

Practical Session Plan

PSCOE-IT

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
20	Explanation of Experiment	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
10	Preparation of Algorithm and logic development for program	Paper work	Introduces, Facilitates, Explains	Discuss	Knowledge, Communication, intrapersonal, Application
60	Coding of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication,
					intrapersonal, Application

20	Testing of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

Title: Classification with Multilayer Perceptron using Scikit-learn (MNIST Dataset)

Objectives: To Study Classification with Multilayer Perceptron using Scikit-Learn (MNIST Dataset).

Aim/ Problem Statement**: Implement a multi-layer perceptron to classify the MNIST data**

Use MLP Classifier in sklearn.

Apparatus: Computer

Theory: Theory:

Procedure/Algorithm :

Source Code:

Implement a multi-layer perceptron to classify the MNIST data that we have been working with all semester. Use MLPClassifier in sklearn.

In [1]:

```
from scipy.stats import mode
import numpy as np
#from mnist import MNIST
from time import time
import pandas as pd
```

RSOE-IT

```
import os
import matplotlib.pyplot as matplot
import matplotlib
%matplotlib inline

import random
matplot.rcdefaults()
from time import time
from IPython.display import display, HTML
from itertools import chain
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
import seaborn as sb
from sklearn.model_selection import ParameterGrid
from sklearn.neural_network import MLPClassifier
import warnings
warnings.filterwarnings('ignore')
```

In [2]:

```
from tensorflow.examples.tutorials.mnist import input_data
mnist = input_data.read_data_sets('MNIST_data/')
```

```
Extracting MNIST_data/train-images-idx3-ubyte.gz
Extracting MNIST_data/train-labels-idx1-ubyte.gz
Extracting MNIST_data/t10k-images-idx3-ubyte.gz
Extracting MNIST_data/t10k-labels-idx1-ubyte.gz
```

In [3]:

```
train = mnist.train.images
validation = mnist.validation.images
test = mnist.test.images

trlab = mnist.train.labels
vallab = mnist.validation.labels
tslab = mnist.test.labels

train = np.concatenate((train, validation), axis=0)
trlab = np.concatenate((trlab, vallab), axis=0)
```

In [17]:

```
mlp = MLPClassifier()
mlp.fit(train, trlab)
```

Out[17]:

```
MLPClassifier(activation='relu', alpha=0.0001, batch_size='auto', beta_1=0.9,
       beta_2=0.999, early_stopping=False, epsilon=1e-08,
       hidden_layer_sizes=(100,), learning_rate='constant',
       learning_rate_init=0.001, max_iter=200, momentum=0.9,
       nesterovs_momentum=True, power_t=0.5, random_state=None,
```

RSCOE-IT

```
    shuffle=True, solver='adam', tol=0.0001, validation_fraction=0.1,
    verbose=False, warm_start=False)
```

In [19]:

```
accuracy_score(tslab, mlp.predict(test)) # Test Accuracy
```

Out[19]:

0.9778

(a) Find a good combination of parameter values for the MLPClassifier that provides the best accuracy on the 10,000 test images. Describe your parameter choices and why you believe these values are good choices.

We choose Alpha and Max_iter as the parameter to run the model on and select the best from those.

According to Scikit Learn- MLP classfier documentation, Alpha is L2 or ridge penalty (regularization term) parameter. Max_iter is Maximum number of iterations, the solver iterates until convergence. So, these ones makes sense as we could see some changing values of accuracy while tuning them,

In [4]:

```
i = 0
df = pd.DataFrame(columns = ['alpha','max_iter','train_acc','test_acc','train_time'])
for a in [0.00001,0.0001,0.001,0.01, 0.1, 1, 10]:
   for mi in [10,100,200,500,1000,2000]:
      st = time()
      mlp = MLPClassifier(alpha=a, max_iter=mi)
      mlp.fit(train, trlab)
      end = time() - st

      acc_tr = accuracy_score(trlab, mlp.predict(train)) # Train Accuracy
      acc = accuracy_score(tslab, mlp.predict(test)) # Test Accuracy
      df.loc[i] = [a,mi,acc_tr,acc,end]
      i=i+1
```

In [5]:

```
df # Results
```

Out[5]:

	Alpha	max_iter	train_acc	test_acc	train_time
0	**0.00001**	**10.0**	**0.989533**	**0.9751**	**9.513362**
1	**0.00001**	**100.0**	**1.000000**	**0.9792**	**35.246778**
2	**0.00001**	**200.0**	**0.999950**	**0.9782**	**35.320454**
3	**0.00001**	**500.0**	**1.000000**	**0.9787**	**42.534170**

RSCOE-IT

4	**0.00001**	**1000.0**	**0.999650**	**0.9764**	**35.734075**
5	**0.00001**	**2000.0**	**0.999983**	**0.9781**	**39.468011**
6	**0.00010**	**10.0**	**0.989417**	**0.9758**	**9.325813**
7	**0.00010**	**100.0**	**0.999900**	**0.9782**	**35.416230**
8	**0.00010**	**200.0**	0.992400	**0.9709**	**44.222664**
9	**0.00010**	**500.0**	**0.998633**	**0.9764**	**40.425065**
10	**0.00010**	**1000.0**	**0.994300**	**0.9728**	**38.569622**
11	**0.00010**	**2000.0**	**0.999983**	**0.9776**	**43.423535**
12	**0.00100**	**10.0**	**0.988950**	**0.9743**	**9.373941**
13	**0.00100**	**100.0**	**0.999950**	**0.9781**	**33.523193**
14	**0.00100**	**200.0**	**0.999917**	**0.9795**	**31.724409**
15	**0.00100**	**500.0**	**0.999633**	**0.9752**	**42.274478**
16	**0.00100**	**1000.0**	**0.999400**	**0.9778**	**31.284236**
17	**0.00100**	**2000.0**	**1.000000**	**0.9782**	**37.767680**
18	**0.01000**	**10.0**	**0.987850**	**0.9761**	**9.285706**
19	**0.01000**	**100.0**	**0.999000**	**0.9779**	**36.310609**
20	**0.01000**	**200.0**	**0.998700**	**0.9771**	**43.446597**
21	**0.01000**	**500**	**0.999167**	**0.9779**	**36.770835**
22	**0.01000**	**1000.0**	**0.999500**	**0.9796**	**40.686252**
23	**0.01000**	**2000.0**	**0.997367**	**0.9775**	**36.911208**
24	**0.10000**	**10.0**	**0.983750**	**0.9731**	**9.193460**
25	**0.10000**	**100.0**	**0.990917**	**0.9787**	**32.303950**
26	**0.10000**	**200.0**	**0.992667**	**0.9802**	**33.043918**

RSCOE-IT

27	**0.10000**	**500.0**	**0.991667**	**0.9787**	**37.553917**
28	**0.10000**	**1000.0**	**0.992283**	**0.9779**	**33.461008**
29	**0.10000**	**2000.0**	**0.991833**	**0.9797**	**36.753789**
30	**1.00000**	**10.0**	**0.953550**	**0.9536**	**9.187445**
31	**1.00000**	**100.0**	**0.957850**	**0.9568**	**25.442695**
32	**1.00000**	**200.0**	**0.959267**	**0.9581**	**26.905587**
33	**1.00000**	**500.0**	**0.957533**	**0.9601**	**24.393637**
34	**1.00000**	**1000.0**	**0.958333**	**0.9589**	**25.306217**
35	**1.00000**	**2000.0**	**0.958250**	**0.9568**	**43.656156**
36	**10.00000**	**10.0**	**0.882517**	**0.8880**	**9.103221**
37	**10.00000**	**100.0**	**0.877850**	**0.8825**	**20.827414**
38	**10.00000**	**200.0**	**0.881017**	**0.8848**	**22.719449**
39	**10.00000**	**500.0**	**0.882133**	**0.8890**	**15.773970**
40	**10.00000**	**1000.0**	**0.880600**	**0.8878**	**15.246566**
41	**10.00000**	**2000.0**	**0.883083**	**0.8910**	**24.275589**

PSCOE-IT

(b) Using the parameters you found in part (a) present a plot that compares the test accuracy vs. the number of nodes in the hidden layer. Comment on your results.

As we can see that Model number 26 gives the best testing accuracy so parameters we select are
Parameters:
Alpha = 0.1
Max Iterations = 200
In [7]:

```
acc = []
acc_tr = []
timelog = []
for l in [10,20,50,100,200,500,1000]:
    t = time()
    mlp = MLPClassifier(alpha=0.1, max_iter=200, hidden_layer_sizes=(l,))
    mlp.fit(train, trlab)
    endt = time() - t

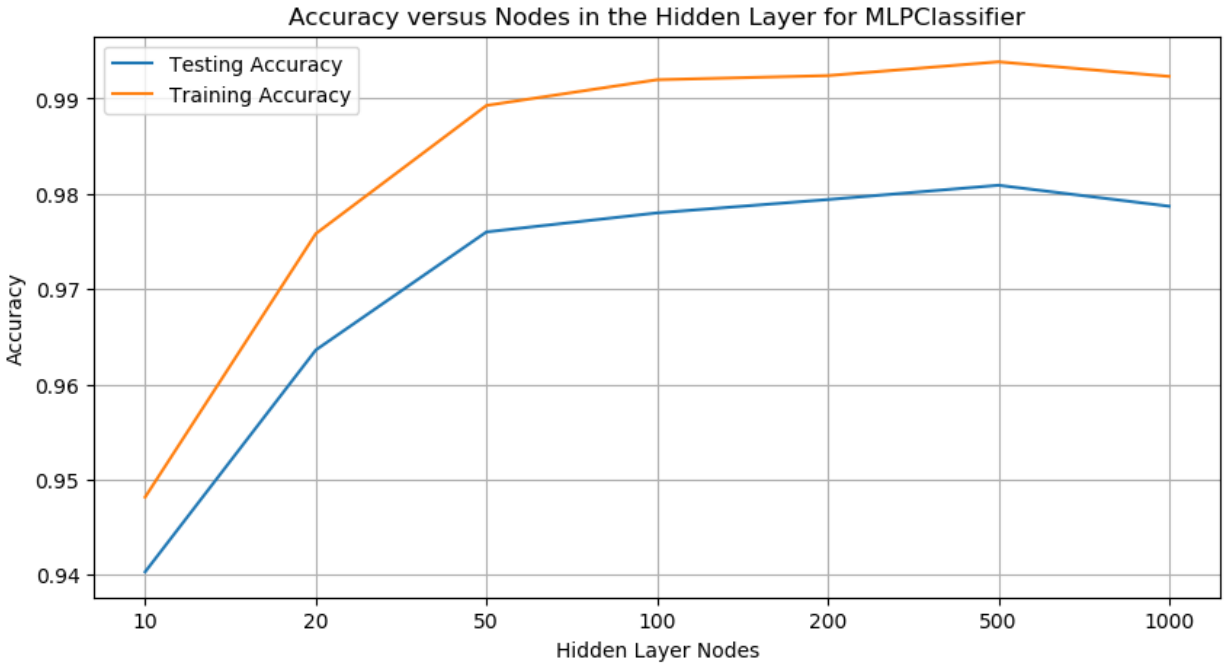
    a_tr = accuracy_score(trlab, mlp.predict(train)) # Train Accuracy
```

```
    a = accuracy_score(tslab, mlp.predict(test)) # Test Accuracy

    acc_tr.append(a_tr)
    acc.append(a)
    timelog.append(endt)
```

In [15]:

```
l = [10,20,50,100,200,500,1000]
N = len(l)
l2 = np.arange(N)
matplot.subplots(figsize=(10, 5))
matplot.plot(l2, acc, label="Testing Accuracy")
matplot.plot(l2, acc_tr, label="Training Accuracy")
matplot.xticks(l2,l)
matplot.grid(True)
matplot.xlabel("Hidden Layer Nodes")
matplot.ylabel("Accuracy")
matplot.legend()
matplot.title('Accuracy versus Nodes in the Hidden Layer for MLPClassifier', fontsize=12)
matplot.show()
```



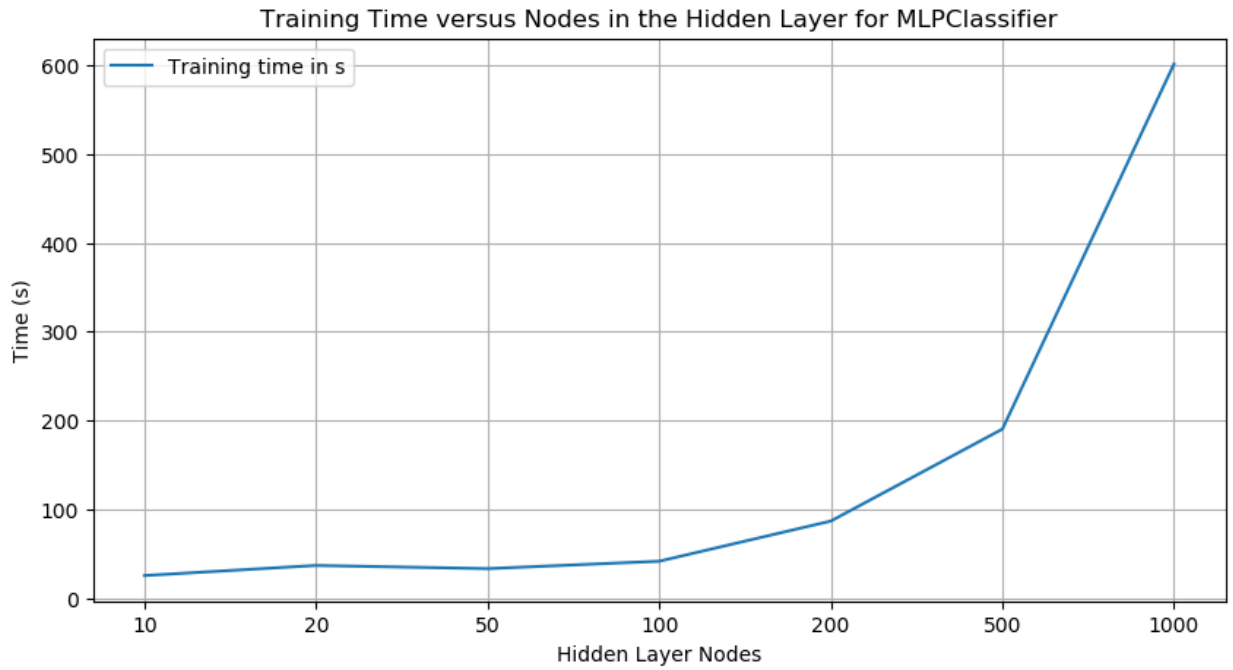
Inferences from the graph:

- So as we go from left to right on this graph, the bias decreases and variance increases
- As the nodes increases there is an increase in accuracy
- That stops at 500 nodes, then we see overfitting kicking in
- The accuracy drops after the nodes increases above 500 in the hidden layer
- Giving us the optimum nodes = 500 for this problem

- From the below graph, we can also see that as the nodes increases naturally the time increases as well

In [16]:

```
l = [10,20,50,100,200,500,1000]
N = len(l)
l2 = np.arange(N)
matplot.subplots(figsize=(10, 5))
matplot.plot(l2, timelog, label="Training time in s")
matplot.xticks(l2,l)
matplot.grid(True)
matplot.xlabel("Hidden Layer Nodes")
matplot.ylabel("Time (s)")
matplot.legend()
matplot.title('Training Time versus Nodes in the Hidden Layer for MLPClassifier', fontsize=12)
matplot.show()
```



Final model parameters for highest test accuracy:
Alpha = 0.1
Max Iterations = 200
Hidden Layer Nodes = 500

(c) How does the accuracy of your MLP classifier compare to what you found with KNN, Naïve Bayes, Logistic Regression, and SVM on this data set? How does the training time of the MLP classifier compare to the others?

Accuracy listed belongs to the best parameters in each model.

Model	**Accuracy**	**Training Time**
MLP	~98.2 %	~ 180 s
kNN	~97.17 %	24 minutes
NB	81.49 %	12 minutes
Log Reg	~87.77 %	2 hrs
SVM	~96.4 %	18 minutes

*Note: the training time to be taken with a grain of salt as all the times reported from the previous models were run on different systems and are either the average or best case times for those models.

Regardless of the time inconsistencies in measuring, MLP is still miles ahead of all the other models in terms of magnitude of time and also in Accuracy.
Observations:

Sample Calculations: Explained in Algorithm.

Expected Input: Real Dataset

Expected Output: Graph

Conclusion:

MLP Classifier performs the best and most optimally, giving us the best test accuracy i.e. ***98.2%*** and it trains within 3 minutes/180 seconds.

---------- **X** ----------

RSCOE-IT

EXPERIMENT NO.2

Classification of MNIST Dataset using CNN

FashionMNIST classification using LeNet-5 architecture

Practical Session Plan.

RSCOE-IT

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
20	Explanation of Experiment	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
10	Preparation of Classification of MNIST Dataset using CNN	Paper work	Introduces, Facilitates, Explains	Discuss	Knowledge, Communication, intrapersonal, Application
60	Coding of FashionMNIST classification	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication,
	using LeNet-5 architecture .				intrapersonal, Application

20	Testing of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

RCOE-IT

Title:

Classification of MNIST Dataset using CNN

Objectives: To Study and Implement Classification of MNIST Dataset using CNN

Aim/ Problem Statement: Fashion MNIST classification using LeNet-5 architecture.

Apparatus: Computer

Theory:

At a high-level perspective, the LeNet-5 comprises of 3 parts namely; (i) two convolutional layers and (ii) a dense block consisting of three convolutional layers and sometimes uses sub-sampling or pooling layers between these layers. As for the training technique of this algorithm, each time an image is supplied to the network, the image is broken down into small slices known as local receptor fields. Since each neuron/unit in a layer is connected to neurons in the previous layer, the neurons can then extract visual elementary features from these local receptor fields such as edges and corners. These features are the combined in the latter layers of the network to in order to map/form higher-order features. The LeNet model was trained on the regular MNIST data set comprised of 60,000 images, of which 50,000 were used for training and 10,000 for testing using the mean squared error as the loss function for 20 iterations. The network achieved an error rate of

0.9%. In our task, we will be applying LeNet-5 to the Fashion MNIST data, but first, let's import some dependencies!

```
# import dependencies
import numpy as np # for scientific computation
import pandas as pd
import torch
import torchvision
from torch import nn # building blocks of a CNN
import matplotlib.pyplot as plt # ploting library
from torchvision.transforms import Resize, ToTensor , Normalize, Compose
import torch.optim as optim # optimizer algorithms package
from conf_mat import plot_confusion_matrix
import torch.nn.functional as F
import os
import time
from sklearn.metrics import confusion_matrix
torch.manual_seed(seed=59) # set seed for reproducibility
# def helper functions
@torch.no_grad()
def get_all_preds(data_loader, model):
    """function to return all predicted probabilities"""
    all_preds = torch.tensor([]) # init empty tensor
    for batch in data_loader:
        imgs, lbls = batch
        preds = model(imgs)
        all_preds = torch.cat((all_preds, preds), dim=0)
    return all_preds


def get_correct(preds, lbls):
    """function tells us how many predictions are correct"""
    return preds.argmax(1).eq(lbls).sum().item()


# prepare data & specify some transforms
batch_size = 100
transf = {
    "train":Compose([ToTensor(),
                     Normalize(mean=0.3814, std=0.3994)]),
     "val":                          Compose([ToTensor(),
Normalize(mean=0.3814, std=0.3994)])
            }
root_dir = "../../OOP"
```

ROOT-IT

```
                                        train_data =
                                        torchvision.datasets.FashionMNIST(root=root_dir,
                                        train=True,
                                                                transform=transf['train'],
                                                                download=True )
                                        val_data =
                                        torchvision.datasets.FashionMNIST(root=root_dir,
                                        train=False,
                                                                transform=transf['val'],
                                                                download=True )
                                        # train dataloaders
                                        train_loader =
                                        torch.utils.data.DataLoader(dataset=train_data,
                                        batch_size=batch_size, shuffle=True, num_workers=1)
                                        val_loader =
                                        torch.utils.data.DataLoader(dataset=val_data,
                                        batch_size=batch_size, shuffle=True, num_workers=1)
                                        # test dataloaders
                                        train_test_loader =
                                        torch.utils.data.DataLoader(dataset=train_data,
                                        batch_size=batch_size, shuffle=False, num_workers=1)
                                        val_test_loader =
                                        torch.utils.data.DataLoader(dataset=val_data,
                                        batch_size=batch_size, shuffle=False, num_workers=1)
# define LeNet-5
                        lenet = nn.Sequential()
                        lenet.add_module("conv1",       nn.Conv2d(in_channels=1,       out_channels=6,
                        kernel_size=5, stride=1, padding=2))
                        lenet.add_module("tanh1", nn.Tanh())
                        lenet.add_module("avg_pool1", nn.AvgPool2d(kernel_size=2, stride=2))
                        lenet.add_module("conv2",       nn.Conv2d(in_channels=6,       out_channels=16,
                        kernel_size=5, stride=1))
                        lenet.add_module("avg_pool2", nn.AvgPool2d(kernel_size=2, stride=2))
                        lenet.add_module("conv3",      nn.Conv2d(in_channels=16,      out_channels=120,
                        kernel_size=5,stride=1))
                        lenet.add_module("tanh2", nn.Tanh())
                        lenet.add_module("flatten", nn.Flatten(start_dim=1))
                        lenet.add_module("fc1", nn.Linear(in_features=120 , out_features=84))
                        lenet.add_module("tanh3", nn.Tanh())
                        lenet.add_module("fc2", nn.Linear(in_features=84, out_features=10))
                        print(lenet)
Sequential(
                (conv1): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
                (tanh1): Tanh()
```

RSCOE-IT

```
    (avg_pool1): AvgPool2d(kernel_size=2, stride=2, padding=0)
    (conv2): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
    (avg_pool2): AvgPool2d(kernel_size=2, stride=2, padding=0)
    (conv3): Conv2d(16, 120, kernel_size=(5, 5), stride=(1, 1))
    (tanh2): Tanh()
    (flatten): Flatten()
    (fc1): Linear(in_features=120, out_features=84, bias=True)
    (tanh3): Tanh()
    (fc2): Linear(in_features=84, out_features=10, bias=True)
  )
# visualizing some samples
                    samples = next(iter(train_loader))
                    imgs = samples[0]
                    lbls = samples[1]
                    plt.figure(figsize=(10,10))
                    grid = torchvision.utils.make_grid(tensor=imgs, nrow=10)
                    grid = np.clip(grid,0,1)
                    plt.imshow(X=np.transpose(grid, axes=(1,2,0)), cmap="gray");
```

RSCOE-IT

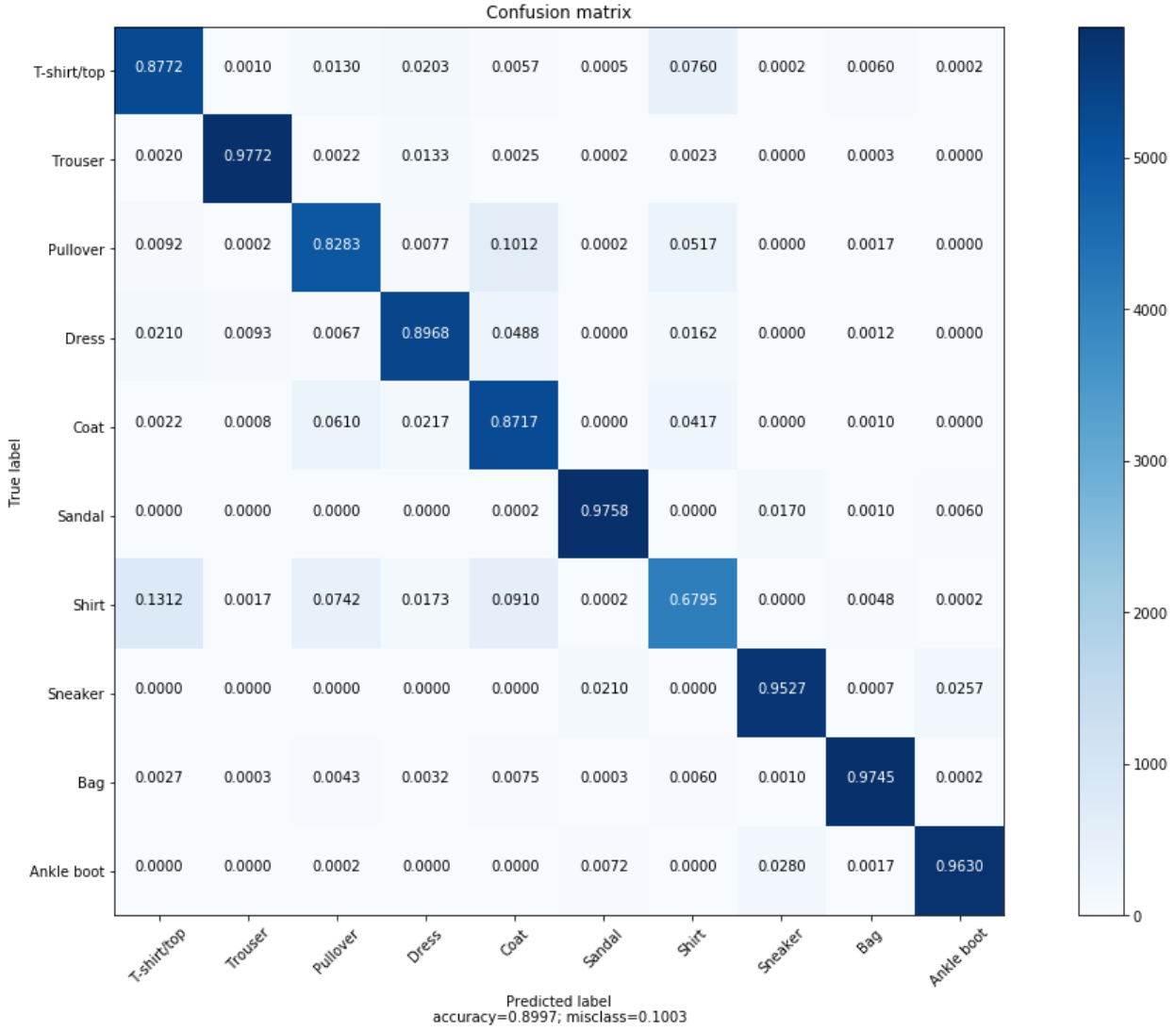



```
# train LeNet-5
                optimizer = optim.Adam(lr=0.001, params=lenet.parameters())
                for epoch in range(5):
                    start_time = time.time()
                    total_loss = 0
                    total_correct = 0
                    for batch in train_loader:
                        imgs, lbls = batch

                        preds = lenet(imgs) # get preds
                        loss = F.cross_entropy(preds, lbls) # cal loss
                        optimizer.zero_grad() # zero gradients
                        loss.backward() # calculates gradients
                        optimizer.step() # update the weights

                        total_loss += loss.item()
```

```
            total_correct += get_correct(preds, lbls)
            accuracy = total_correct/len(train_data)
        end_time = time.time() - start_time
        print("epoch", epoch+1, "| accuracy:", round(accuracy,2),"%", "|
    epoch_duration:",end_time,"sec","| total_loss:", total_loss)
# preds
    probs = get_all_preds(data_loader=train_test_loader, model=lenet) # validation on train data
    tags = train_data.targets
    cm = confusion_matrix(y_true=tags, y_pred=probs.argmax(1))
    plot_confusion_matrix(cm, target_names=train_data.classes, normalize=True)
```



Conclusion :- We have Studied Classification of MNIST Dataset using CNN.
Expected Input / Output / Result : Real data Set/Chart, Graph.

EXPERIMENT NO.3

Face recognition using CNN

Practical Session Plan

RSCOE-IT

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
20	Explanation of Experiment	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
10	Preparation of Algorithm and logic development for program	Paper work	Introduces, Facilitates, Explains	Discuss	Knowledge, Communication, intrapersonal, Application
60	Coding of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication,

					intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

SCOE-IT

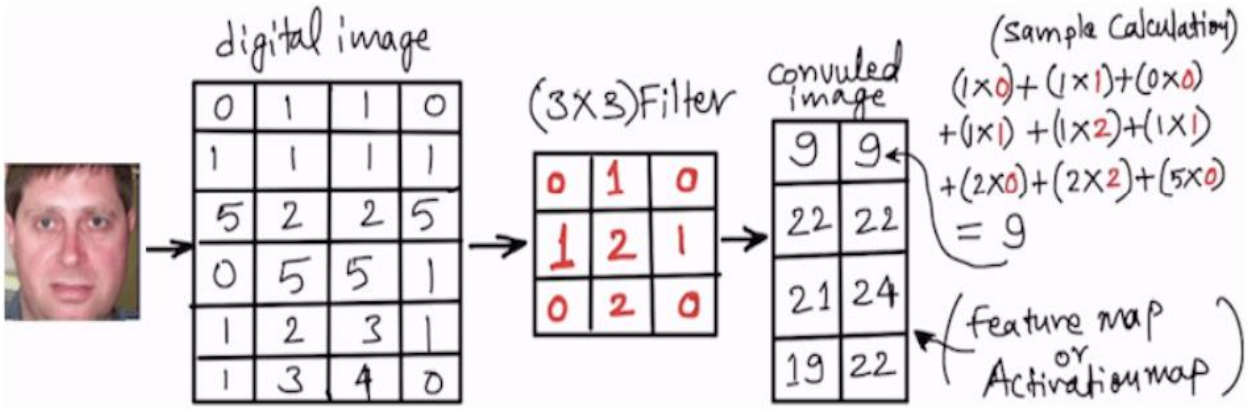
Title: Face recognition using CNN

Objectives: To study Face recognition using CNN

Aim/ Problem Statement**:** Face Recognition using Deep Learning CNN in Python

Apparatus: Computer

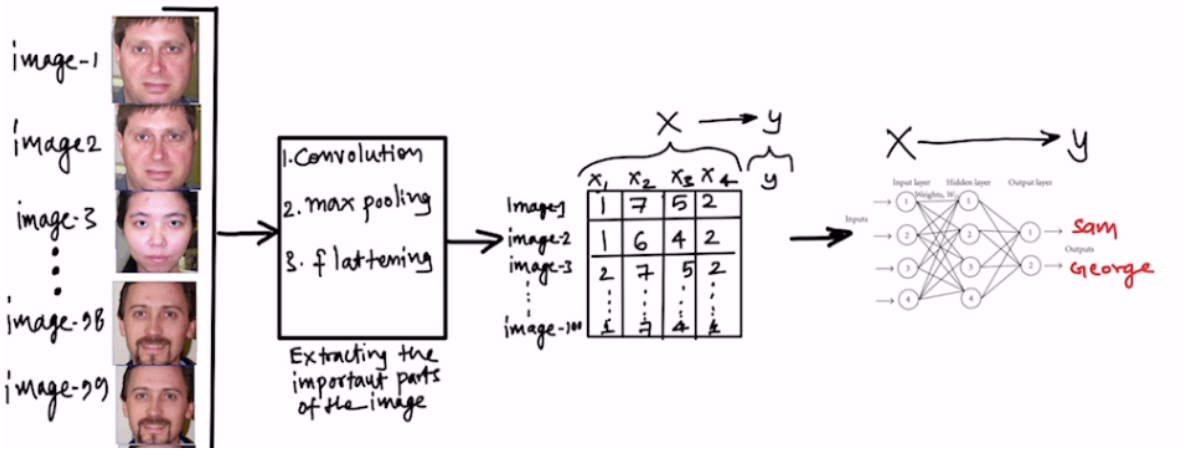
Theory:



The Convolution step in CNN

Convolutional Neural Networks(CNN) changed the way we used to learn images. It made it very very easy! CNN mimics the way humans see images, by focussing on one portion of the image at a time and scanning the whole image.

CNN boils down every image as a vector of numbers, which can be learned by the fully connected Dense layers of ANN.
Below diagram summarises the overall flow of CNN algorithm.



In this case study, We will show you how to implement a face recognition model using CNN. You can use this template to create an image classification model on any group of images by putting them in a folder and creating a class.

The data contains cropped face images of 16 people divided into Training and testing. We will train the CNN model using the images in the Training folder and then test the model by using the unseen images from the testing folder, to check if the model is able to recognise the face number of the unseen images or not.

```
# Deep Learning CNN model to recognize face
'''This script uses a database of images and creates CNN model on top of it to test
   if the given image is recognized correctly or not'''

'''####### IMAGE PRE-PROCESSING for TRAINING and TESTING data #######'''

# Specifying the folder where images are present
TrainingImagePath='/Users/farukh/Python Case Studies/Face Images/Final Training Images'

from keras.preprocessing.image import ImageDataGenerator
# Understand more about ImageDataGenerator at below link
#    https://blog.keras.io/building-powerful-image-classification-models-using-very-little-data.html

# Defining pre-processing transformations on raw images of training data
# These hyper parameters helps to generate slightly twisted versions
# of the original image, which leads to a better model, since it learns
# on the good and bad mix of images
train_datagen = ImageDataGenerator(
        shear_range=0.1,
```

```
        zoom_range=0.1,
        horizontal_flip=True)

# Defining pre-processing transformations on raw images of testing data
# No transformations are done on the testing images
test_datagen = ImageDataGenerator()

# Generating the Training Data
training_set = train_datagen.flow_from_directory(
        TrainingImagePath,
        target_size=(64, 64),
        batch_size=32,
        class_mode='categorical')

# Generating the Testing Data
test_set = test_datagen.flow_from_directory(
        TrainingImagePath,
        target_size=(64, 64),
        batch_size=32,
        class_mode='categorical')

# Printing class labels for each face
test_set.class_indices
```

```
Found 244 images belonging to 16 classes.
Found 244 images belonging to 16 classes.

{'face1': 0,
 'face10': 1,
 'face11': 2,
 'face12': 3,
 'face13': 4,
 'face14': 5,
 'face15': 6,
 'face16': 7,
 'face2': 8,
 'face3': 9,
 'face4': 10,
 'face5': 11,
 'face6': 12,
 'face7': 13,
 'face8': 14,
 'face9': 15}
```

Creating a mapping for index and face names

The above class_index dictionary has face names as keys and the numeric mapping as values. We need to swap it, because the classifier model will return the answer as the numeric mapping and we need to get the face-name out of it.

Also, since this is a multi-class classification problem, we are counting the number of unique faces, as that will be used as the number of output neurons in the output layer of fully connected ANN classifier.

```
'''############ Creating lookup table for all faces ############'''
# class_indices have the numeric tag for each face
TrainClasses=training_set.class_indices

# Storing the face and the numeric tag for future reference
ResultMap={}
for faceValue,faceName in zip(TrainClasses.values(),TrainClasses.keys()):
    ResultMap[faceValue]=faceName

# Saving the face map for future reference
import pickle
with open("ResultsMap.pkl", 'wb') as fileWriteStream:
    pickle.dump(ResultMap, fileWriteStream)

# The model will give answer as a numeric tag
# This mapping will help to get the corresponding face name for it
print("Mapping of Face and its ID",ResultMap)

# The number of neurons for the output layer is equal to the number of faces
OutputNeurons=len(ResultMap)
print('\n The Number of output neurons: ', OutputNeurons)
```

```
Mapping of Face and its ID {0: 'face1', 1: 'face10', 2: 'face11', 3: 'face12', 4: 'face13', 5: 'face
14', 6: 'face15', 7: 'face16', 8: 'face2', 9: 'face3', 10: 'face4', 11: 'face5', 12: 'face6', 13: 'f
ace7', 14: 'face8', 15: 'face9'}

 The Number of output neurons:  16
```

Creating the CNN face recognition model

In the below code snippet, I have created a CNN model with

- 2 hidden layers of convolution
- 2 hidden layers of max pooling
- 1 layer of flattening
- 1 Hidden ANN layer
- 1 output layer with 16-neurons (one for each face)

You can increase or decrease the convolution, max pooling, and hidden ANN layers and the number of neurons in it.

PSCOE-IT

Just keep in mind, the more layers/neurons you add, the slower the model becomes.

Also, when you have large amount of images, in the tune of 50K and above, then your laptop' CPU might not be efficient to learn those many images. You will have to get a GPU enabled laptop, or use cloud services like AWS or Google Cloud.

Since the data we have used for the demonstration is small containing only 244 images for training, you can run it on your laptop easily ☺

Apart from selecting the best number of layers and the number of neurons in it, for each layer, there are some hyper parameters which needs to be tuned as well.

Take a quick look at some of the important hyperparameters

- **Filters**=32: This number indicates how many filters we are using to look at the image pixels during the convolution step. Some filters may catch sharp edges, some filters may catch color variations some filters may catch outlines, etc. In the end, we get important information from the images. In the first layer the number of filters=32 is commonly used, then increasing the power of 2. Like in the next layer it is 64, in the next layer, it is 128 so on and so forth.
- **kernel_size=(5,5)**: This indicates the size of the sliding window during convolution, in this case study we are using 5X5 pixels sliding window.
- **strides=(1, 1):** How fast or slow should the sliding window move during convolution. We are using the lowest setting of 1X1 pixels. Means slide the convolution window of 5X5 (kernal_size) by 1 pixel in the x-axis and 1 pixel in the y-axis until the whole image is scanned.
- **input_shape=(64,64,3):** Images are nothing but matrix of RGB color codes. during our data pre-processing we have compressed the images to 64X64, hence the expected shape is 64X64X3. Means 3 arrays of 64X64, one for RGB colors each.
- **kernel_initializer='uniform'**: When the Neurons start their computation, some algorithm has to decide the value for each weight. This parameter specifies that. You can choose different values for it like 'normal' or 'glorot_uniform'.
- **activation='relu'**: This specifies the activation function for the calculations inside each neuron. You can choose values like 'relu', 'tanh', 'sigmoid', etc.
- **optimizer='adam':** This parameter helps to find the optimum values of each weight in the neural network. 'adam' is one of the most useful optimizers, another one is 'rmsprop'
- **batch_size=10**: This specifies how many rows will be passed to the Network in one go after which the SSE calculation will begin and the neural network will start adjusting its weights based on the errors.
  When all the rows are passed in the batches of 10 rows each as specified in this parameter, then we call that 1-epoch. Or one full data cycle. This is also known as mini-batch gradient descent. A small value of batch_size will make the LSTM look at the data slowly, like 2 rows at a time or 4 rows at a time which could lead to overfitting, as compared to a large value like 20 or 50 rows at a time, which will make the LSTM look at the data fast which could lead to underfitting. Hence a proper value must be chosen using hyperparameter tuning.

- **Epochs=10**: The same activity of adjusting weights continues for 10 times, as specified by this parameter. In simple terms, the LSTM looks at the full training data 10 times and adjusts its weights.

```
'''######################### Create CNN deep learning model ########################'''
from keras.models import Sequential
from keras.layers import Convolution2D
from keras.layers import MaxPool2D
from keras.layers import Flatten
from keras.layers import Dense

'''Initializing the Convolutional Neural Network'''
classifier= Sequential()

''' STEP--1 Convolution
# Adding the first layer of CNN
# we are using the format (64,64,3) because we are using TensorFlow backend
# It means 3 matrix of size (64X64) pixels representing Red, Green and Blue components of pixels
'''
classifier.add(Convolution2D(32, kernel_size=(5, 5), strides=(1, 1), input_shape=(64,64,3), activation='relu'))

'''# STEP--2 MAX Pooling'''
classifier.add(MaxPool2D(pool_size=(2,2)))

'''############### ADDITIONAL LAYER of CONVOLUTION for better accuracy #################'''
classifier.add(Convolution2D(64, kernel_size=(5, 5), strides=(1, 1), activation='relu'))

classifier.add(MaxPool2D(pool_size=(2,2)))

'''# STEP--3 FLattening'''
classifier.add(Flatten())

'''# STEP--4 Fully Connected Neural Network'''
classifier.add(Dense(64, activation='relu'))

classifier.add(Dense(OutputNeurons, activation='softmax'))

'''# Compiling the CNN'''
#classifier.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
classifier.compile(loss='categorical_crossentropy', optimizer = 'adam', metrics=["accuracy"])

############################################################
import time
# Measuring the time taken by the model to train
StartTime=time.time()

# Starting the model training
classifier.fit_generator(
```

RSCOE-IT

```
        training_set,
        steps_per_epoch=30,
        epochs=10,
        validation_data=test_set,
        validation_steps=10)

EndTime=time.time()
print("###### Total Time Taken: ", round((EndTime-StartTime)/60), 'Minutes ######')
```

```
2.7025 - val_accuracy: 0.9318
Epoch 6/10
30/30 [==============================] - 9s 294ms/step - loss: 0.0161 - accuracy: 0.9967 - val_loss:
2.6716 - val_accuracy: 0.9545
Epoch 7/10
30/30 [==============================] - 9s 295ms/step - loss: 0.0383 - accuracy: 0.9890 - val_loss:
2.6862 - val_accuracy: 0.9545
Epoch 8/10
30/30 [==============================] - 9s 297ms/step - loss: 0.0306 - accuracy: 0.9870 - val_loss:
2.6709 - val_accuracy: 0.9392
Epoch 9/10
30/30 [==============================] - 8s 278ms/step - loss: 0.0313 - accuracy: 0.9923 - val_loss:
2.6518 - val_accuracy: 0.9903
Epoch 10/10
30/30 [==============================] - 9s 294ms/step - loss: 0.0155 - accuracy: 0.9946 - val_loss:
2.6692 - val_accuracy: 0.9708
############## Total Time Taken:  1 Minutes ############
```

Testing the CNN classifier on unseen images

Using any one of the images from the testing data folder, we can check if the model is able to recognize the face.

```
'''########### Making single predictions ###########'''
import numpy as np
from keras.preprocessing import image

ImagePath='/Users/farukh/Python Case Studies/Face Images/Final Testing
Images/face4/3face4.jpg'
test_image=image.load_img(ImagePath,target_size=(64, 64))
test_image=image.img_to_array(test_image)

test_image=np.expand_dims(test_image,axis=0)

result=classifier.predict(test_image,verbose=0)
#print(training_set.class_indices)

print('####'*10)
print('Prediction is: ',ResultMap[np.argmax(result)])
```

```
########################################
Prediction is:  face4
```

The model has predicted this face correctly! You can try for other faces and see if it gets recognized. You can also add your own pics and train the model again.

Observations:

Sample Calculations: Explained in Algorithm.

Expected Input : Real data Set

Expected Output/Results: Face recognition

RSCOE-IT

EXPERIMENT NO.4

Time Series Prediction using RNN

Practical Session Plan

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
20	Explanation of Experiment	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
10	Preparation of Algorithm and logic development for program	Paper work	Introduces, Facilitates, Explains	Discuss	Knowledge, Communication, intrapersonal, Application
60	Coding of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

Title: Time Series Prediction using RNN

RSCOE-IT

Objectives: To study Time Series Prediction using RNN.

Aim/ Problem Statement: Time Series Analysis with LSTM using Python's keras Library

Apparatus: Computer

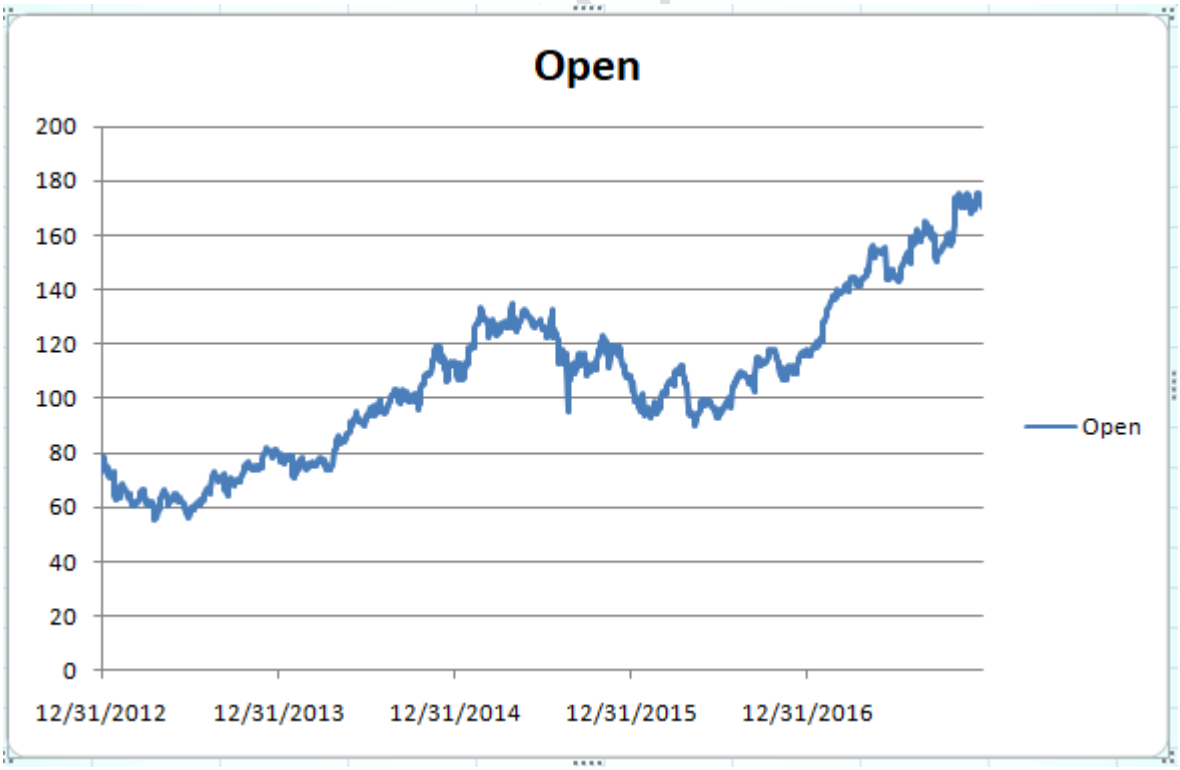
Theory: Time series analysis refers to the analysis of change in the trend of the data over a period of time. Time series analysis has a variety of applications. One such application is the prediction of the future value of an item based on its past values. Future stock price prediction is probably the best example of such an application

Dataset

The data that we are going to use for this article can be downloaded from Yahoo Finance. For training our algorithm, we will be using the Apple stock prices from 1st January 2013 to 31 December 2017. For the sake of prediction, we will use the Apple stock prices for the month of January 2018. So in order to evaluate the performance of the algorithm, download the actual stock prices for the month of January 2018 as well.

Let's now see how our data looks. Open the Apple stock price training file that contains data for five years. You will see that it contains seven columns: Date, Open, High, Low, Close, Adj Close and Volume. We will be predicting the opening stock price, therefore we are not interested in the rest of the columns.

If you plot the opening stock prices against the date, you will see the following plot:



You can see that the trend is highly non-linear and it is very difficult to capture the trend using this information. This is where the power of LSTM can be utilized. LSTM (Long Short-Term Memory

network) is a type of recurrent neural network capable of remembering the past information and while predicting the future values, it takes this past information into account.

Enough of the preliminaries, let's see how LSTM can be used for time series analysis.

Predicting Future Stock Prices

Stock price prediction is similar to any other machine learning problem where we are given a set of features and we have to predict a corresponding value. We will perform the same steps as we do perform in order to solve any machine learning problem. Follow these steps:

Import Libraries

The first step, as always is to import the required libraries. Execute the following script to do so:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Import Dataset

Execute the following script to import the data set. For the sake of this article, the data has been stored in the Datasets folder, inside the "E" drive. You can change the path accordingly.

```
apple_training_complete = pd.read_csv(r'E:\Datasets\apple_training.csv')
```

As we said earlier, we are only interested in the opening price of the stock. Therefore, we will filter all the data from our training set and will retain only the values for the **Open** column. Execute the following script:

```
apple_training_processed = apple_training_complete.iloc[:, 1:2].values
```

Data Normalization

As a rule of thumb, whenever you use a neural network, you should normalize or scale your data. We will use `MinMaxScaler` class from the `sklear.preprocessing` library to scale our data between 0 and 1. The `feature_range` parameter is used to specify the range of the scaled data. Execute the following script:

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range = (0, 1))

apple_training_scaled = scaler.fit_transform(apple_training_processed)
```

Convert Training Data to Right Shape

As I said earlier, in a time series problems, we have to predict a value at time T, based on the data from days T-N where N can be any number of steps. In this article, we are going to predict the opening stock price of the data based on the opening stock prices for the past 60 days. I have tried and tested different numbers and found that the best results are obtained when past 60 time steps are used. You can try different numbers and see how your algorithm performs.

Our feature set should contain the opening stock price values for the past 60 days while the label or dependent variable should be the stock price at the 61st day. Execute the following script to create feature and label set.

```
features_set = []
labels = []
for i in range(60, 1260):
    features_set.append(apple_training_scaled[i-60:i, 0])
    labels.append(apple_training_scaled[i, 0])
```

In the script above we create two lists: `feature_set` and `labels`. There are 1260 records in the training data. We execute a loop that starts from 61st record and stores all the previous 60 records to the `feature_set` list. The 61st record is stored in the `labels` list.

We need to convert both the `feature_set` and the `labels` list to the numpy array before we can use it for training. Execute the following script:

```
features_set, labels = np.array(features_set), np.array(labels)
```

In order to train LSTM on our data, we need to convert our data into the shape accepted by the LSTM. We need to convert our data into three-dimensional format. The first dimension is the number of records or rows in the dataset which is 1260 in our case. The second dimension is the number of time steps which is 60 while the last dimension is the number of indicators. Since we are only using one feature, i.e **Open**, the number of indicators will be one. Execute the following script:

```
features_set = np.reshape(features_set, (features_set.shape[0], features_set.shape[1], 1))
```

Training The LSTM

We have preprocessed our data and have converted it into the desired format. now is the time to create our LSTM. The LSTM model that we are going to create will be a sequential model with multiple layers. We will add four LSTM layers to our model followed by a dense layer that predicts the future stock price.

Let's first import the libraries that we are going to need in order to create our model:

```
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LSTM
from keras.layers import Dropout
```

In the script above we imported the `Sequential` class from `keras.models` library and `Dense`, `LSTM`, and `Dropout` classes from `keras.layers` library.

As a first step, we need to instantiate the `Sequential` class. This will be our model class and we will add LSTM, Dropout and Dense layers to this model. Execute the following script

```
model = Sequential()
```

Creating LSTM and Dropout Layers

Let's add LSTM layer to the model that we just created. Execute the following script to do so:

```
model.add(LSTM(units=50, return_sequences= True, input_shape=(features_set.shape[ 1], 1)))
```

To add a layer to the sequential model, the `add` method is used. Inside the `add` method, we passed our LSTM layer. The first parameter to the LSTM layer is the number of neurons or nodes that we want in the layer. The second parameter is `return_sequences`, which is set to true since we will add more layers to the model. The first parameter to the `input_shape` is the number of time steps while the last parameter is the number of indicators.

Let's now add a dropout layer to our model. Dropout layer is added to avoid over-fitting, which is a phenomenon where a machine learning model performs better on the training data compared to the test data. Execute the following script to add dropout layer.

```
model.add(Dropout(0.2))
```

Let's add three more LSTM and dropout layers to our model. Run the following script.

```
model.add(LSTM(units= 50 , return_sequences= True ))
model.add(Dropout( 0.2 ))

model.add(LSTM(units= 50 , return_sequences= True ))
model.add(Dropout( 0.2 ))

model.add(LSTM(units= 50 ))
```

```
model.add(Dropout( 0.2 ))
```

Creating Dense Layer

To make our model more robust, we add a dense layer at the end of the model. The number of neurons in the dense layer will be set to 1 since we want to predict a single value in the output.

```
model.add(Dense(units = 1 ))
```

Model Compilation

Finally, we need to compile our LSTM before we can train it on the training data. The following script compiles the our model.

```
model. compile (optimizer = 'adam' , loss = 'mean_squared_error' )
```

We call the compile method on the Sequential model object which is "model" in our case. We use the mean squared error as loss function and to reduce the loss or to optimize the algorithm, we use the adam optimizer.

Algorithm Training

Now is the time to train the model that we defined in the previous few steps. To do so, we call the `fit` method on the `model` and pass it our training features and labels as shown below:

```
model.fit(features_set, labels, epochs = 100 , batch_size = 32 )
```

Depending upon your hardware, model training can take some time.

Testing our LSTM

We have successfully trained our LSTM, now is the time to test the performance of our algorithm on the test set by predicting the opening stock prices for the month of January 2018. However, as we did with the training data, we need to convert our test data in the right format.

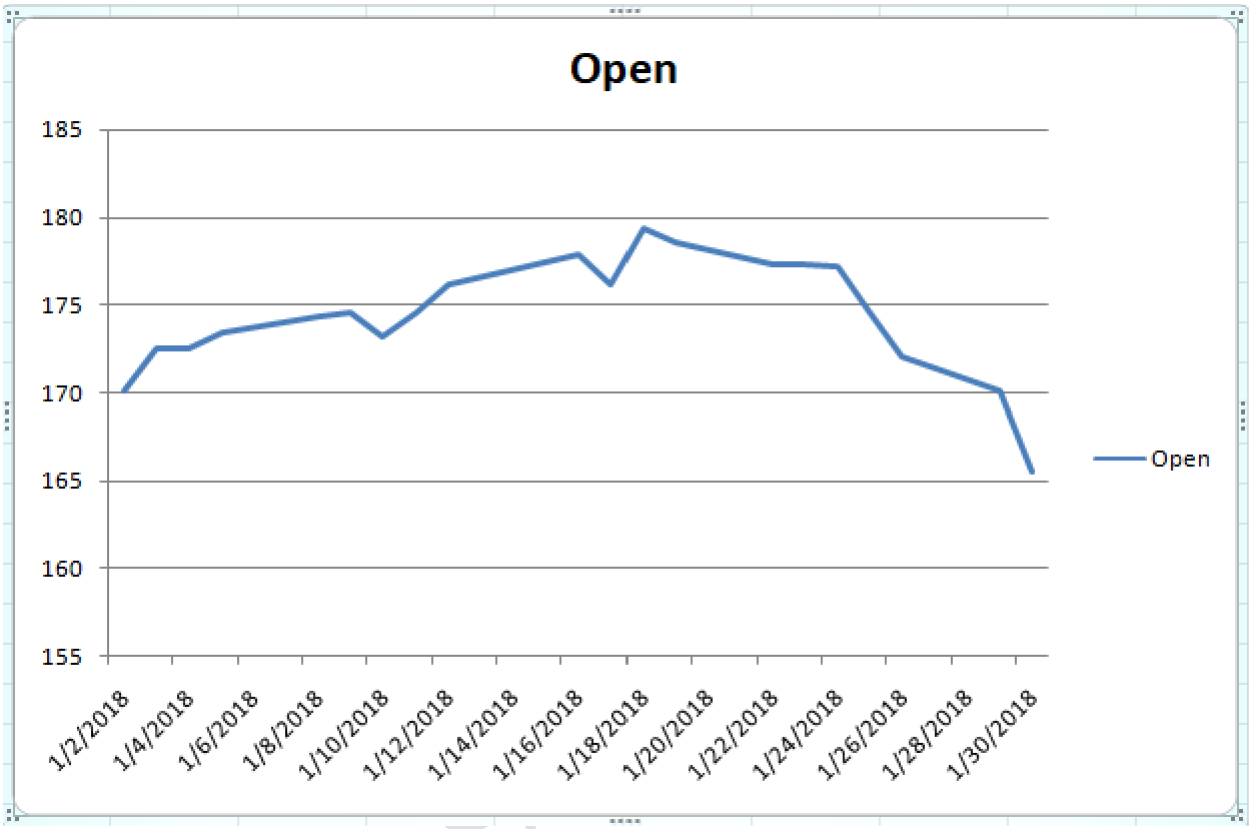
Let's first import our test data. Execute the following script:

```
apple_testing_complete = pd.read_csv( r'E:\Datasets\apple_testing.csv' )
apple_testing_processed = apple_testing_complete.iloc[:, 1 : 2 ].values
```

PSCOE-IT

In the above script, we import our test data and as we did with the training data, we removed all the columns from the test data except the column that contains opening stock prices.

If the opening stock prices for the month of January 2018 are plotted against the dates, you should see the following graph.



You can see that the trend is highly non-linear. Overall, the stock prices see small rise at the start of the month followed by a downward trend at the end of the month, with a slight increase and decrease in the stock prices in-between. It is extremely difficult to forecast such a trend. Let's see if the LSTM we trained is actually able to predict such a trend.

Converting Test Data to Right Format

For each day of January 2018, we want our feature set to contain the opening stock prices for the previous 60 days. For the 1st of January, we need the stock prices for the previous 60 days. To do so, we need to concatenate our training data and test data before preprocessing. Execute the following script to do so:

```
apple_total = pd.concat((apple_training_complete['Open'], apple_testing_complete['Open']),
axis=0)
```

Now let's prepare our test inputs. The input for each day should contain the opening stock prices for the previous 60 days. That means we need opening stock prices for the 20 test days for the month of January 2018 and the 60 stock prices from the last 60 days for the training set. Execute the following script to fetch those 80 values.

```
test_inputs = apple_total[len(apple_total) - len(apple_testing_complete) - 60:].values
```

As we did for the training set, we need to scale our test data. Execute the following script:

```
test_inputs = test_inputs.reshape(-1,1)
test_inputs = scaler.transform(test_inputs)
```

We scaled our data, now let's prepare our final test input set that will contain previous 60 stock prices for the month of January. Execute the following script:

```
test_features = []
for i in range(60, 80):
    test_features.append(test_inputs[i-60:i, 0])
```

Finally, we need to convert our data into the three-dimensional format which can be used as input to the LSTM. Execute the following script:

```
test_features = np.array(test_features)
test_features = np.reshape(test_features, (test_features.shape[0], test_features.shape[1], 1))
```

Making Predictions

Now is the time to see the magic. We preprocessed our test data and now we can use it to make predictions. To do so, we simply need to call the `predict` method on the model that we trained. Execute the following script:

```
predictions = model.predict(test_features)
```

Since we scaled our data, the predictions made by the LSTM are also scaled. We need to reverse the scaled prediction back to their actual values. To do so, we can use the `inverse_transform` method of the scaler object we created during training. Take a look at the following script:

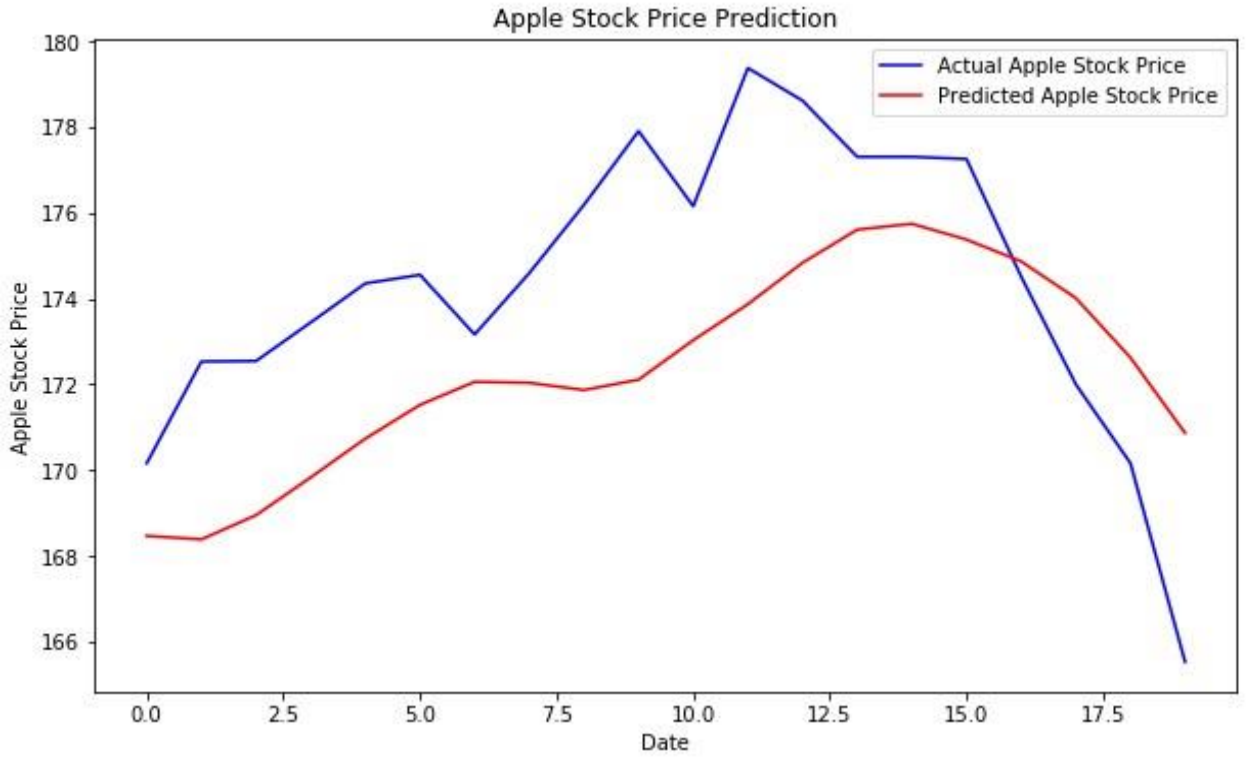
```
predictions = scaler.inverse_transform(predictions)
```

RSCOE-IT

Finally, let's see how well did our algorithm predicted the future stock prices. Execute the following script:

```
plt.figure(figsize=(10,6))
plt.plot(apple_testing_processed, color= 'blue', label='Actual Apple Stock Price')
plt.plot(predictions , color='red', label='Predicted Apple Stock Price')
plt.title('Apple Stock Price Prediction')
plt.xlabel('Date')
plt.ylabel('Apple Stock Price')
plt.legend()
plt.show()
```

The output looks like this:



In the output, the blue line represents the actual stock prices for the month of January 2018, while the red line represents the predicted stock prices. You can clearly see that our algorithm has been able to capture the overall trend.

Conclusion

A long short-term memory network (LSTM) is one of the most commonly used neural networks for time series analysis. Thus We have studied **Time Series Prediction using RNN**

Upon completion of Experiment students will be able to:

1. Analyze the Real data Set and Observing the charts, graphs.

RSCOE-IT

EXPERIMENT NO.5

Image generation using GAN

Practical Session Plan

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
20	Explanation of Experiment	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
10	Preparation of Algorithm for Image generation using GAN	Paper work	Introduces, Facilitates, Explains	Discuss	Knowledge, Communication, intrapersonal, Application
60	Coding of image generation program.	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application

RSCOE-IT

10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

Title: Image generation using GAN

Objectives: To demonstrate Image generation using GAN.

Aim/ Problem Statement: To Analyze and differentiate fake and real image through GAN

Apparatus: Computer

Theory:

CIFAR-10 Small Object Photograph Dataset

CIFAR is an acronym that stands for the Canadian Institute For Advanced Research and the CIFAR-10 dataset was developed along with the CIFAR-100 dataset (covered in the next section) by researchers at the CIFAR institute.
The dataset is comprised of 60,000 32×32 pixel color photographs of objects from 10 classes, such as frogs, birds, cats, ships, airplanes, etc.

These are very small images, much smaller than a typical photograph, and the dataset is intended for computer vision research.

Keras provides access to the CIFAR10 dataset via the cifar10.load_dataset() function. It returns two tuples, one with the input and output elements for the standard training dataset, and another with the input and output elements for the standard test dataset.
The example below loads the dataset and summarizes the shape of the loaded dataset.

Note: the first time you load the dataset, Keras will automatically download a compressed version of the images and save them under your home directory in *~/.keras/datasets/*. The download is fast as the dataset is only about 163 megabytes in its compressed form.

```
# example of loading the cifar10 dataset
from keras.datasets.cifar10 import load_data
# load the images into memory
(trainX, trainy), (testX, testy) = load_data()
# summarize the shape of the dataset
```

```
print('Train', trainX.shape, trainy.shape)
print('Test', testX.shape, testy.shape)
```

Running the example loads the dataset and prints the shape of the input and output components of the train and test splits of images.

We can see that there are 50K examples in the training set and 10K in the test set and that each image is a square of 32 by 32 pixels.

```
Train (50000, 32, 32, 3) (50000, 1)
Test (10000, 32, 32, 3) (10000, 1)
```

The images are color with the object centered in the middle of the frame.

We can plot some of the images from the training dataset with the matplotlib library using the imshow() function.

```
# plot raw pixel data
pyplot.imshow(trainX[i])
```

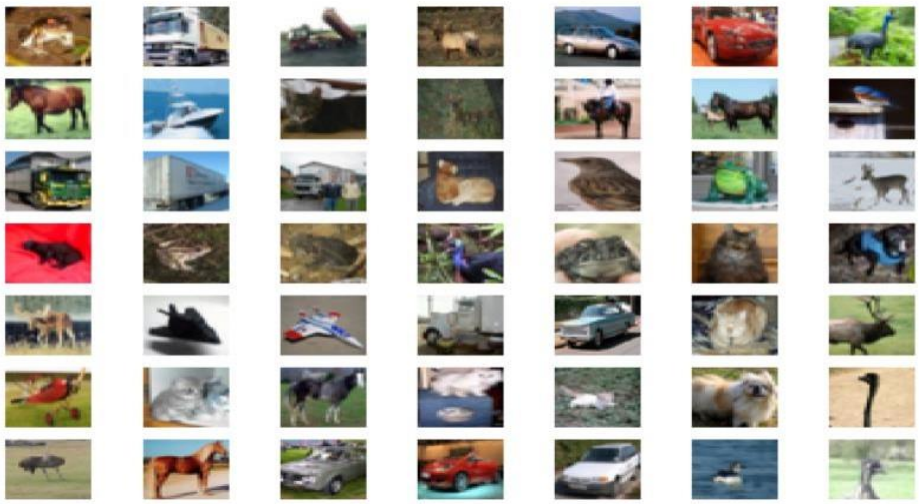
The example below plots the first 49 images from the training dataset in a 7 by 7 square.

```
# example of loading and plotting the cifar10 dataset
from keras.datasets.cifar10 import load_data
from matplotlib import pyplot
# load the images into memory
(trainX, trainy), (testX, testy) = load_data()
# plot images from the training dataset
for i in range(49):
	# define subplot
	pyplot.subplot(7, 7, 1 + i)
	# turn off axis
	pyplot.axis('off')
	# plot raw pixel data
	pyplot.imshow(trainX[i])
pyplot.show()
```

Running the example creates a figure with a plot of 49 images from the CIFAR10 training dataset, arranged in a 7×7 square.

In the plot, you can see small photographs of planes, trucks, horses, cars, frogs, and so on.

RSCOE-IT



Plot of the First 49 Small Object Photographs From the CIFAR10 Dataset.

We will use the images in the training dataset as the basis for training a Generative Adversarial Network.

Specifically, the generator model will learn how to generate new plausible photographs of objects using a discriminator that will try and distinguish between real images from the CIFAR10 training dataset and new images output by the generator model.

This is a non-trivial problem that requires modest generator and discriminator models that are probably most effectively trained on GPU hardware.

For help using cheap Amazon EC2 instances to train deep learning models, see the post:

- How to Setup Amazon AWS EC2 GPUs to Train Keras Deep Learning Models (step-by-step)

How to Define and Train the Discriminator Model

The first step is to define the discriminator model.

The model must take a sample image from our dataset as input and output a classification prediction as to whether the sample is real or fake. This is a binary classification problem.

- **Inputs**: Image with three color channel and 32×32 pixels in size.
- **Outputs**: Binary classification, likelihood the sample is real (or fake).

The discriminator model has a normal convolutional layer followed by three convolutional layers using a stride of 2×2 to downsample the input image. The model has no pooling layers and a single node in the output layer with the sigmoid activation function to predict whether the input sample is real or fake. The model is trained to minimize the binary cross entropy loss function, appropriate for binary classification.

We will use some best practices in defining the discriminator model, such as the use of LeakyReLU instead of ReLU, using Dropout, and using the Adam version of stochastic gradient descent with a learning rate of 0.0002 and a momentum of 0.5.

The *define_discriminator()* function below defines the discriminator model and parametrizes the size of the input image.

```
# define the standalone discriminator model
def define_discriminator(in_shape=(32,32,3)):
	model = Sequential()
	# normal
	model.add(Conv2D(64, (3,3), padding='same', input_shape=in_shape))
	model.add(LeakyReLU(alpha=0.2))
	# downsample
	model.add(Conv2D(128, (3,3), strides=(2,2), padding='same'))
	model.add(LeakyReLU(alpha=0.2))
	# downsample
	model.add(Conv2D(128, (3,3), strides=(2,2), padding='same'))
	model.add(LeakyReLU(alpha=0.2))
	# downsample
	model.add(Conv2D(256, (3,3), strides=(2,2), padding='same'))
	model.add(LeakyReLU(alpha=0.2))
	# classifier
	model.add(Flatten())
	model.add(Dropout(0.4))
	model.add(Dense(1, activation='sigmoid'))
	# compile model
	opt = Adam(lr=0.0002, beta_1=0.5)
	model.compile(loss='binary_crossentropy', optimizer=opt, metrics=['accuracy'])
	return model
```

We can use this function to define the discriminator model and summarize it.

The complete example is listed below.

```
# example of defining the discriminator model
from keras.models import Sequential
from keras.optimizers import Adam
from keras.layers import Dense
from keras.layers import Conv2D
from keras.layers import Flatten
from keras.layers import Dropout
from keras.layers import LeakyReLU
```

RSCOE-IT

```
from keras.utils.vis_utils import plot_model

# define the standalone discriminator model
def define_discriminator(in_shape=(32,32,3)):
	model = Sequential()
	# normal
	model.add(Conv2D(64, (3,3), padding='same', input_shape=in_shape))
	model.add(LeakyReLU(alpha=0.2))
	# downsample
	model.add(Conv2D(128, (3,3), strides=(2,2), padding='same'))
	model.add(LeakyReLU(alpha=0.2))
	# downsample
	model.add(Conv2D(128, (3,3), strides=(2,2), padding='same'))
	model.add(LeakyReLU(alpha=0.2))
	# downsample
	model.add(Conv2D(256, (3,3), strides=(2,2), padding='same'))
	model.add(LeakyReLU(alpha=0.2))
	# classifier
	model.add(Flatten())
	model.add(Dropout(0.4))
	model.add(Dense(1, activation='sigmoid'))
	# compile model
	opt = Adam(lr=0.0002, beta_1=0.5)
	model.compile(loss='binary_crossentropy', optimizer=opt, metrics=['accuracy'])
	return model

# define model
model = define_discriminator()
# summarize the model
model.summary()
# plot the model
plot_model(model, to_file='discriminator_plot.png', show_shapes=True,
show_layer_names=True)
```

Running the example first summarizes the model architecture, showing the input and output from each layer.

We can see that the aggressive 2×2 stride acts to down-sample the input image, first from 32×32 to 16×16, then to 8×8 and more before the model makes an output prediction.

This pattern is by design as we do not use pooling layers and use the large stride to achieve a similar downsampling effect. We will see a similar pattern, but in reverse in the generator model in the next section.

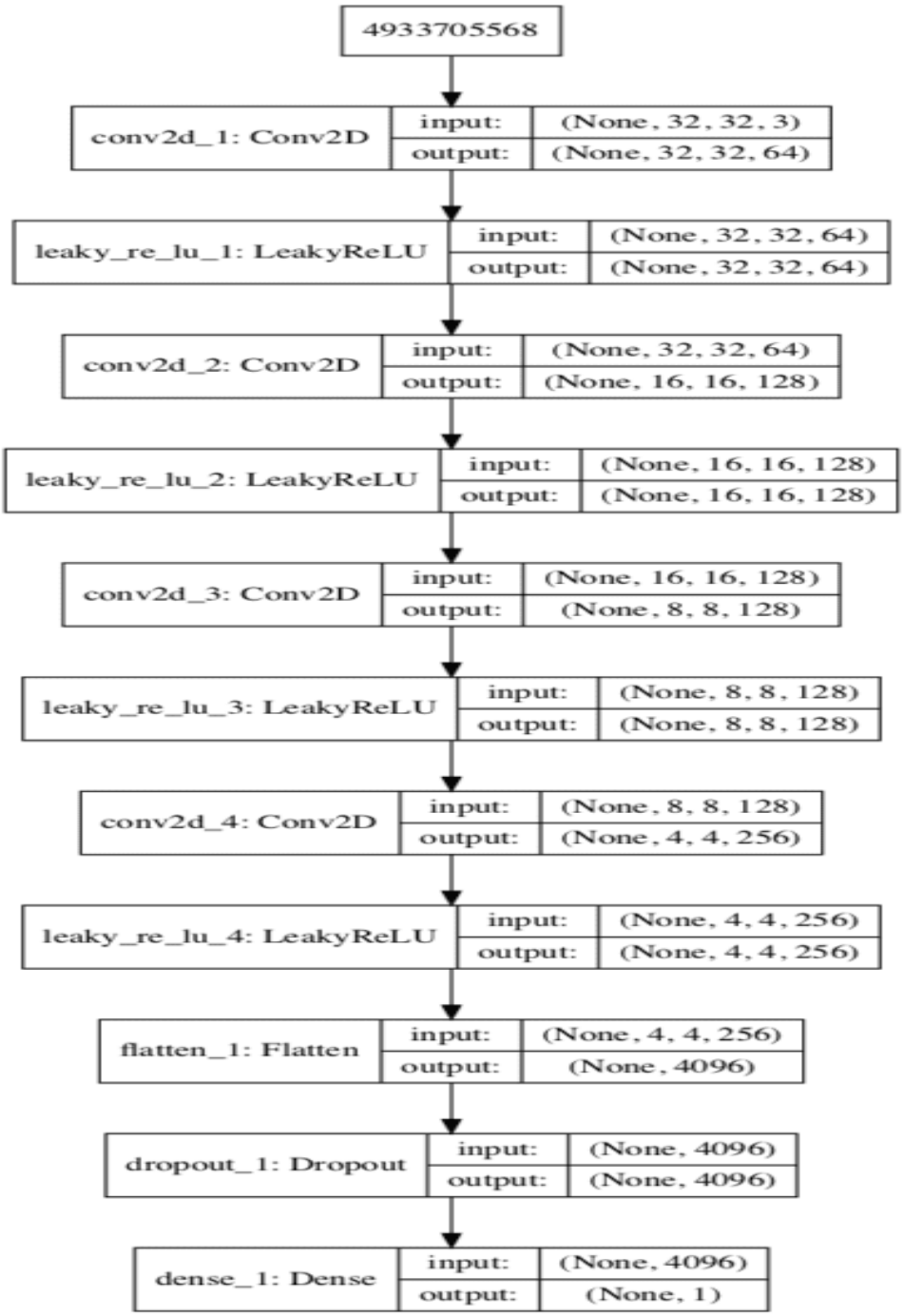
```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_1 (Conv2D)            (None, 32, 32, 64)        1792
```

RSCOE-IT

```
_________________________________________________________________
leaky_re_lu_1 (LeakyReLU)    (None, 32, 32, 64)        0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 16, 16, 128)       73856
_________________________________________________________________
leaky_re_lu_2 (LeakyReLU)    (None, 16, 16, 128)       0
_________________________________________________________________
conv2d_3 (Conv2D)            (None, 8, 8, 128)         147584
_________________________________________________________________
leaky_re_lu_3 (LeakyReLU)    (None, 8, 8, 128)         0
_________________________________________________________________
conv2d_4 (Conv2D)            (None, 4, 4, 256)         295168
_________________________________________________________________
leaky_re_lu_4 (LeakyReLU)    (None, 4, 4, 256)         0
_________________________________________________________________
flatten_1 (Flatten)          (None, 4096)              0
_________________________________________________________________
dropout_1 (Dropout)          (None, 4096)              0
_________________________________________________________________
dense_1 (Dense)              (None, 1)                 4097
=================================================================
Total params: 522,497
Trainable params: 522,497
Non-trainable params: 0
_________________________________________________________________
```

A plot of the model is also created and we can see that the model expects a vector input and will predict a single output.

Note: creating this plot assumes that the pydot and graphviz libraries are installed. If this is a problem, you can comment out the import statement and the call to the *plot_model()* function.



Plot of the Discriminator Model in the CIFAR10 Generative Adversarial Network

We could start training this model now with real examples with a class label of one and randomly generate samples with a class label of zero.

The development of these elements will be useful later, and it helps to see that the discriminator is just a normal neural network model for binary classification.

First, we need a function to load and prepare the dataset of real images.

We will use the *cifar.load_data()* function to load the CIFAR-10 dataset and just use the input part of the training dataset as the real images.

```
...
# load cifar10 dataset
(trainX, _), (_, _) = load_data()
```

We must scale the pixel values from the range of unsigned integers in [0,255] to the normalized range of [-1,1].
The generator model will generate images with pixel values in the range [-1,1] as it will use the tanh activation function, a best practice.

It is also a good practice for the real images to be scaled to the same range.

```
...
# convert from unsigned ints to floats
X = trainX.astype('float32')
# scale from [0,255] to [-1,1]
X = (X - 127.5) / 127.5
```

The *load_real_samples()* function below implements the loading and scaling of real CIFAR-10 photographs.

```
# load and prepare cifar10 training images
def load_real_samples():
	# load cifar10 dataset
	(trainX, _), (_, _) = load_data()
	# convert from unsigned ints to floats
	X = trainX.astype('float32')
	# scale from [0,255] to [-1,1]
	X = (X - 127.5) / 127.5
	return X
```

The model will be updated in batches, specifically with a collection of real samples and a collection of generated samples. On training, an epoch is defined as one pass through the entire training dataset.

We could systematically enumerate all samples in the training dataset, and that is a good approach, but good training via stochastic gradient descent requires that the training dataset be shuffled prior to each epoch. A simpler approach is to select random samples of images from the training dataset.

The *generate_real_samples()* function below will take the training dataset as an argument and will select a random subsample of images; it will also return class labels for the sample, specifically a class label of 1, to indicate real images.

```
# select real samples
```

RSCOE-IT

```
def generate_real_samples(dataset, n_samples):
	# choose random instances
	ix = randint(0, dataset.shape[0], n_samples)
	# retrieve selected images
	X = dataset[ix]
	# generate 'real' class labels (1)
	y = ones((n_samples, 1))
	return X, y
```

Now, we need a source of fake images.

We don't have a generator model yet, so instead, we can generate images comprised of random pixel values, specifically random pixel values in the range [0,1], then scaled to the range [-1, 1] like our scaled real images.

The *generate_fake_samples()* function below implements this behavior and generates images of random pixel values and their associated class label of 0, for fake.

```
# generate n fake samples with class labels
def generate_fake_samples(n_samples):
	# generate uniform random numbers in [0,1]
	X = rand(32 * 32 * 3 * n_samples)
	# update to have the range [-1, 1]
	X = -1 + X * 2
	# reshape into a batch of color images
	X = X.reshape((n_samples, 32, 32, 3))
	# generate 'fake' class labels (0)
	y = zeros((n_samples, 1))
	return X, y
```

Finally, we need to train the discriminator model.

This involves repeatedly retrieving samples of real images and samples of generated images and updating the model for a fixed number of iterations.

We will ignore the idea of epochs for now (e.g. complete passes through the training dataset) and fit the discriminator model for a fixed number of batches. The model will learn to discriminate between real and fake (randomly generated) images rapidly, therefore not many batches will be required before it learns to discriminate perfectly.

The *train_discriminator()* function implements this, using a batch size of 128 images, where 64 are real and 64 are fake each iteration.

We update the discriminator separately for real and fake examples so that we can calculate the accuracy of the model on each sample prior to the update. This gives insight into how the discriminator model is performing over time.

SCOE-IT

```
# train the discriminator model
def train_discriminator(model, dataset, n_iter=20, n_batch=128):
	half_batch = int(n_batch / 2)
	# manually enumerate epochs
	for i in range(n_iter):
		# get randomly selected 'real' samples
		X_real, y_real = generate_real_samples(dataset, half_batch)
		# update discriminator on real samples
		_, real_acc = model.train_on_batch(X_real, y_real)
		# generate 'fake' examples
		X_fake, y_fake = generate_fake_samples(half_batch)
		# update discriminator on fake samples
		_, fake_acc = model.train_on_batch(X_fake, y_fake)
		# summarize performance
		print('>%d real=%.0f%% fake=%.0f%%' % (i+1, real_acc*100, fake_acc*100))
```

Tying all of this together, the complete example of training an instance of the discriminator model on real and randomly generated (fake) images is listed below.

```
# example of training the discriminator model on real and random cifar10 images
from numpy import expand_dims
from numpy import ones
from numpy import zeros
from numpy.random import rand
from numpy.random import randint
from keras.datasets.cifar10 import load_data
from keras.optimizers import Adam
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Conv2D
from keras.layers import Flatten
from keras.layers import Dropout
from keras.layers import LeakyReLU

# define the standalone discriminator model
def define_discriminator(in_shape=(32,32,3)):
	model = Sequential()
	# normal
	model.add(Conv2D(64, (3,3), padding='same', input_shape=in_shape))
	model.add(LeakyReLU(alpha=0.2))
	# downsample
	model.add(Conv2D(128, (3,3), strides=(2,2), padding='same'))
	model.add(LeakyReLU(alpha=0.2))
	# downsample
	model.add(Conv2D(128, (3,3), strides=(2,2), padding='same'))
	model.add(LeakyReLU(alpha=0.2))
	# downsample
	model.add(Conv2D(256, (3,3), strides=(2,2), padding='same'))
	model.add(LeakyReLU(alpha=0.2))
	# classifier
```

RSCOE-IT

```
        model.add(Flatten())
        model.add(Dropout(0.4))
        model.add(Dense(1, activation='sigmoid'))
        # compile model
        opt = Adam(lr=0.0002, beta_1=0.5)
        model.compile(loss='binary_crossentropy', optimizer=opt, metrics=['accuracy'])
        return model

# load and prepare cifar10 training images
def load_real_samples():
        # load cifar10 dataset
        (trainX, _), (_, _) = load_data()
        # convert from unsigned ints to floats
        X = trainX.astype('float32')
        # scale from [0,255] to [-1,1]
        X = (X - 127.5) / 127.5
        return X

# select real samples
def generate_real_samples(dataset, n_samples):
        # choose random instances
        ix = randint(0, dataset.shape[0], n_samples)
        # retrieve selected images
        X = dataset[ix]
        # generate 'real' class labels (1)
        y = ones((n_samples, 1))
        return X, y

# generate n fake samples with class labels
def generate_fake_samples(n_samples):
        # generate uniform random numbers in [0,1]
        X = rand(32 * 32 * 3 * n_samples)
        # update to have the range [-1, 1]
        X = -1 + X * 2
        # reshape into a batch of color images
        X = X.reshape((n_samples, 32, 32, 3))
        # generate 'fake' class labels (0)
        y = zeros((n_samples, 1))
        return X, y

# train the discriminator model
def train_discriminator(model, dataset, n_iter=20, n_batch=128):
        half_batch = int(n_batch / 2)
        # manually enumerate epochs
        for i in range(n_iter):
                # get randomly selected 'real' samples
                X_real, y_real = generate_real_samples(dataset, half_batch)
                # update discriminator on real samples
                _, real_acc = model.train_on_batch(X_real, y_real)
```

RSCOE-IT

```
        # generate 'fake' examples
        X_fake, y_fake = generate_fake_samples(half_batch)
        # update discriminator on fake samples
        _, fake_acc = model.train_on_batch(X_fake, y_fake)
        # summarize performance
        print('>%d real=%.0f%% fake=%.0f%%' % (i+1, real_acc*100, fake_acc*100))

# define the discriminator model
model = define_discriminator()
# load image data
dataset = load_real_samples()
# fit the model
train_discriminator(model, dataset)
```

Running the example first defines the model, loads the CIFAR-10 dataset, then trains the discriminator model.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

In this case, the discriminator model learns to tell the difference between real and randomly generated CIFAR-10 images very quickly, in about 20 batches.

```
...
>16 real=100% fake=100%
>17 real=100% fake=100%
>18 real=98% fake=100%
>19 real=100% fake=100%
>20 real=100% fake=100%
```

Conclusion: - Thus we have studied Image generation using GAN

RSCOE-IT

EXPERIMENT NO.6

Sentiment analysis of text using LSTM.

Practical Session Plan

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
20	Explanation of Experiment	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application

10	Preparation of Algorithm and logic development for program	Paper work	Introduces, Facilitates, Explains	Discuss	Knowledge, Communication, intrapersonal, Application
60	Coding of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

RSCOE-IT

Title: Sentiment analysis of text using LSTM.

Objectives: To Analyze Sentiment analysis of text using LSTM.

Aim/ Problem Statement: Sentiment Analysis using LSTM and GloVe Embeddings

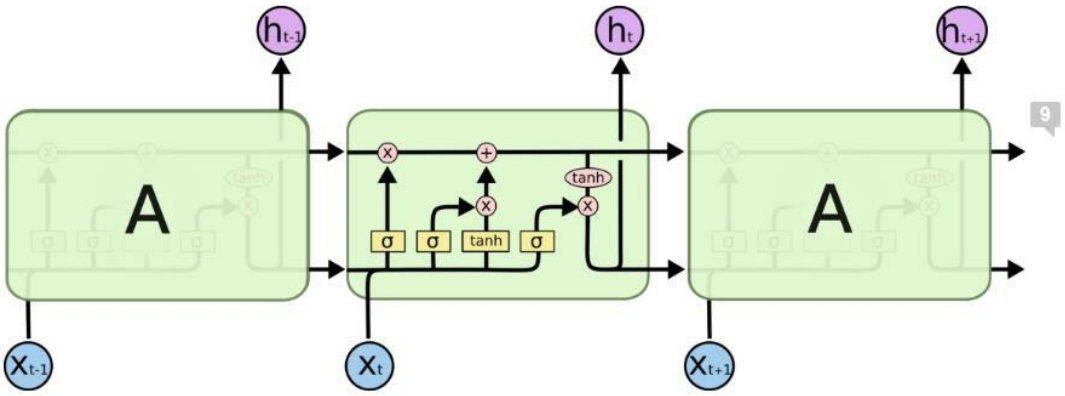
Apparatus: Computer

Theory: Sentiment Analysis has emerged as an important tool for many Organizations and Companies as it gives useful insights into the customer base of a website which in turn helps their decision-making process.

Today I will use the famous ***IMDb Movie Review Dataset*** with 50k movie reviews labeled as 'positive' or 'negative' to implement a simple Sentiment Analysis model using LSTM and GloVe Word Embeddings.

LSTMs

I used ***LSTM(Long Term Short Memory)*** for this classification task, it is a type of RNN architecture(Recurrent Neural Network) which are extensively used nowadays for NLP because it handles long sequence dependencies well.



The repeating module in an LSTM contains four interacting layers.

This is a typical LSTM unit that is repeated over the whole length of a sequence.

GloVe Word Embeddings

GloVe is an unsupervised learning algorithm to learn vector representation i.e word embedding for various words. GloVe stands for Global Vectors for Word Representations. In this code, I will be using the 50-dimensional GloVe vectors for the task at hand.

1. Importing libraries and loading the dataset

First, we will import all the required libraries and packages

```
from keras.datasets import imdb
                                    import pandas as pd
                                    import numpy as np
                                    from keras.layers import LSTM, Activation, Dropout, Dense,
                                    Input
                                    from keras.layers.embeddings import Embedding
                                    from keras.models import Model
                                    import string
                                    import re
                                    from keras.preprocessing.text import Tokenizer
                                    from sklearn.preprocessing import LabelBinarizer
                                    from keras.preprocessing.sequence import pad_sequences
                                    import keras
                                    from sklearn.model_selection import train_test_split
```

Now let us load the dataset into Google Colab (or any platform you use)

```
data = pd.read_csv('/content/drive/My Drive/IMDB
Dataset.csv')
                                                    data['review'] =
                                                    data['review'].str.lower()
```

RSCOE-IT

2. Preprocessing the data

This might be among the most important steps because, the data which we have contains a lot of unnecessary things which can be called noise and it only hinders our algorithm's capability, so we will clean the data a little of this noise.

We will remove stop words from all the movie reviews, now stop words are basically the most commonly occurring words in any language. I have created a list of all the stop words here.

```
stopwords = [ "a", "about", "above", "after", "again", "against", "all", "am", "an",
"and", "any", "are", "as", "at", "be", "because",
                                                    "been",
                                                    "before",
                                                    "being",
                                                    "below",
                                                    "between",
```

"both", "but",
"by",
"could",
"did", "do",
"does",
"doing",
"down",
"during",

"each",
"few", "for",
"from",
"further",
"had", "has",
"have",
"having",
"he", "he'd",
"he'll",
"he's", "her",
"here",

"here's",
"hers",
"herself",
"him",
"himself",
"his", "how",
"how's", "i",
"i'd", "i'll",
"i'm", "i've",
"if", "in",
"into",
"is",
"it", "it's",
"its", "itself",
"let's", "me",
"more",
"most",
"my",
"myself",
"nor", "of",
"on", "once",
"only", "or",

"other",

RSCOE-IT

```
"ought",
"our", "ours",
"ourselves",
"out", "over",
"own",
"same",
"she",
"she'd",
"she'll",
"she's",
"should",
            "so",
"some",
"such",
"than",
"that",
"that's",
"the", "their",
"theirs",
"them",
"themselves"
,          "then",
"there",
"there's",

"these",
"they",
"they'd",
"they'll",
"they're",
"they've",
"this",
"those",
"through",
"to",      "too",
"under",
"until", "up",

"very",
"was",  "we",
"we'd",
"we'll",
"we're",
"we've",
"were",
```

RSCOE-IT

```
"what",
"what's",
"when",
"when's",
"where",
"where's",

"which",
"while",
"who",
"who's",
"whom",
"why",
"why's",
"with",
"would",
"you",
"you'd",
"you'll",
"you're",
"you've",

"your",
"yours",
"yourself",
"yourselves"
]
```

RSCOE-IT

Now let's define some functions to remove the unwanted data

```
def remove_stopwords(data): data['review without stopwords'] = data['review'].apply(lambda x : ' '.join([word for word in x.split() if word not in (stopwords)])) return data def remove_tags(string): result = re.sub('<.*?>',",string) return result data_without_stopwords = remove_stopwords(data)data_without_stopwords['clean_review']= data_without_stopwords['review without stopwords'].apply(lambda cw : remove_tags(cw))data_without_stopwords['clean_review'] = data_without_stopwords['clean_review'].str.replace('[{}]'.format(string.punctuation), ' ')
```

The function ***remove_stopwords*** as the name suggests removes stop words from all the movie reviews, another advantage of removing stop words is that the embedding for many of the stop words like ‘they’ll’ or ‘you’re’ isn't available in GloVe Embeddings so this will ease our process. The second

function ***remove_tags*** is for removing HTML tags which are common in the text, HTML tags look something like <…>. We apply this function using ***lambda*** function. After this, we get the clean reviews which can be used downstream for training.

3. Tokenizing the words and splitting the data

This can be also included in preprocessing but I'm making a different section Now we will make our X and Y sets for train and test purposes, first, we will create lists from the data frame columns for movie reviews and their sentiment.

```
reviews_list = []
                    for i in range(len(reviews)):
                     reviews_list.append(reviews[i])

                    sentiment = data_without_stopwords['sentiment']
```

Now let's convert the sentiment from string to a binary form of 1 and 0, where 1 is for '***positive***' sentiment and 0 for ***'negative'***.

```
y = np.array(list(map(lambda x: 1 if x=="positive" else 0, sentiment))) X_train, X_test,Y_train,
Y_test = train_test_split(reviews_list, y, test_size=0.2, random_state = 45)
```

Now we will tokenize all the words in the train set, so the ***Tokenizer*** basically indexes all the words in the text and each word gets a unique index. This helps in creating dictionary mapping words to an index which will be very necessary to us later.

```
tokenizer                                =
Tokenizer(num_words=5000)
                                          tokenizer.fit_on_texts(X_train)
                                          words_to_index = tokenizer.word_index
```

words_to_index is the dictionary mapping words to their respective index. Now we will write a function to read the contents of the GloVe Vector file, which returns us a dictionary that maps the words to their respective word embeddings.

```
def
read_glove_vector(glove_vec):
                                        with open(glove_vec, 'r', encoding='UTF-8') as f:
```

```
    words = set()
    word_to_vec_map = {}
    for line in f:
      w_line = line.split()
      curr_word = w_line[0]
      word_to_vec_map[curr_word] = np.array(w_line[1:],
dtype=np.float64)
    return word_to_vec_map
```

We define ***maxLen*** which is the maximum length of one review we will use for our training, I have kept the value to be 150, we will pad this later so all the inputs have the same length

```
word_to_vec_map = read_glove_vector('/content/drive/My
Drive/glove.6B.50d.txt')
                                                    maxLen = 150
```

4. Embedding Matrix and setting up the Model

We defined the ***embedding matrix*** here, where all the words which are not in the GloVe dictionary being assigned a zero vector.

```
vocab_len = len(words_to_index)embed_vector_len = word_to_vec_map['moon'].shape[0]
emb_matrix = np.zeros((vocab_len, embed_vector_len)) for word, index in
words_to_index.items(): embedding_vector = word_to_vec_map.get(word) if embedding_vector
is not None: emb_matrix[index, :] = embedding_vector embedding_layer =
Embedding(input_dim=vocab_len, output_dim=embed_vector_len, input_length=maxLen,
weights = [emb_matrix], trainable=False)
```

We also defined the embedding layer using the built-in Keras ***Embedding layer***. The embedding layer maps the words to their embedding vectors from the embedding matrix. We can keep this layer as trainable, which will learn the word embedding itself but as we are using GloVe Embeddings, we won't keep the layer as trainable.

We will use this layer to define our model architecture.

```
def
imdb_rating(input_shape):
    X_indices = Input(input_shape)
    embeddings = embedding_layer(X_indices)
    X = LSTM(128, return_sequences=True)(embeddings)
```

```
X = Dropout(0.6)(X)
X = LSTM(128, return_sequences=True)(X)
X = Dropout(0.6)(X)
X = LSTM(128)(X)
X = Dense(1, activation='sigmoid')(X)
model = Model(inputs=X_indices, outputs=X)
return model
```

This function defines our model architecture, first, we use the embedding layer to map the words to their GloVe vectors, and then those vectors are input to the LSTM layers followed by a Dense layer with 'sigmoid' activation.

5. Training the data and Testing the model

With our model architecture setup, now we will get our data ready for training. Basically, we will convert the reviews in the dataset to their index form by using the ***texts_to_sequences*** function available with tokenizer. After that, we will pad the sequences so all of them have the same length.

```
X_train_indices = tokenizer.texts_to_sequences(X_train)
X_train_indices = pad_sequences(X_train_indices, maxlen=maxLen, padding='post')
```

With everything ready, we will move on to training our data!

```
adam = keras.optimizers.Adam(learning_rate = 0.0001)
model.compile(optimizer=adam, loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train_indices, Y_train, batch_size=64, epochs=15)
```

After training for 15 epochs, I got a reasonable accuracy of 85.29%

```
Epoch 1/15
625/625 [==============================] - 535s 856ms/step - loss: 0.5792 - accuracy: 0.6890
Epoch 2/15
625/625 [==============================] - 536s 858ms/step - loss: 0.4912 - accuracy: 0.7663
Epoch 3/15
625/625 [==============================] - 535s 856ms/step - loss: 0.4715 - accuracy: 0.7778
Epoch 4/15
625/625 [==============================] - 532s 852ms/step - loss: 0.4393 - accuracy: 0.7995
Epoch 5/15
625/625 [==============================] - 534s 855ms/step - loss: 0.4141 - accuracy: 0.8141
Epoch 6/15
625/625 [==============================] - 529s 847ms/step - loss: 0.3913 - accuracy: 0.8253
Epoch 7/15
625/625 [==============================] - 524s 838ms/step - loss: 0.3788 - accuracy: 0.8336
Epoch 8/15
625/625 [==============================] - 520s 832ms/step - loss: 0.3727 - accuracy: 0.8354
Epoch 9/15
625/625 [==============================] - 516s 826ms/step - loss: 0.3643 - accuracy: 0.8416
Epoch 10/15
625/625 [==============================] - 511s 817ms/step - loss: 0.3614 - accuracy: 0.8418
Epoch 11/15
625/625 [==============================] - 524s 839ms/step - loss: 0.3523 - accuracy: 0.8462
Epoch 12/15
625/625 [==============================] - 518s 830ms/step - loss: 0.3485 - accuracy: 0.8480
Epoch 13/15
625/625 [==============================] - 523s 838ms/step - loss: 0.3454 - accuracy: 0.8499
Epoch 14/15
625/625 [==============================] - 519s 830ms/step - loss: 0.3434 - accuracy: 0.8496
Epoch 15/15
625/625 [==============================] - 523s 838ms/step - loss: 0.3382 - accuracy: 0.8529
```

After evaluating the model on the test set, I got an accuracy of 85.08% which is pretty good and there is no problem of overfitting.

```
model.evaluate(X_test_indices, Y_test)
```

```
313/313 [==============================] - 38s 123ms/step - loss: 0.3541 - accuracy: 0.8508
[0.3540983498096466, 0.8507999777793884]
```

Now let's test our model on a random entry in the test set and see if it predicts it correctly.

```
preds = model.predict(X_test_indices)
n = np.random.randint(0,9999)
X_test[n]
if preds[n] > 0.5:
  print('predicted sentiment : positive')
else:
  print('precicted sentiment : negative')
if (Y_test[n] == 1):
  print('correct sentiment : positive')
else:
  print('correct sentiment : negative')
```

```
: ' hey babu riba  film young woman  mariana  nicknamed  esther  famous american movie star  four young men  glenn  sacha  kicha  pop  per
  haps 15 17 years old 1953 belgrade  yugoslavia  five committed friends crazy jazz  blue jeans  anything american seems the close relation
  ship teenagers poignant  ultimately sacrifice willingly made try help one group fallen unexpected difficulties  wake changing communist p
  olitics  go separate ways reunite 1985  the year film made  i enjoyed film reservations  subtitles one thing difficult  especially beginn
  ing  number dialogues no subtitles all  perhaps conversational pace required it  couldn t always read text absorb scene  caused not alway
  s understand character involved  watched movie  a video public library  friend  neither us really understood part story acquiring strepto
  mycin sick relative this yugoslavian coming age film effectively conveyed teenagers  sense invulnerability  idealism  strong loyal bonds
  other  main flashforward  intriguing  keeping guessing end characters vis a vis 1953 cast  actually happened i rate 7 10  like see films
  director  jovan acin  1941 1991  '
```

```
: if preds[n] > 0.5:
    print('predicted sentiment : positive')
  else:
    print('precicted sentiment : negative')

  if (Y_test[n] == 1):
    print('correct sentiment : positive')
  else:
    print('correct sentiment : negative')
```

```
predicted sentiment : positive
correct sentiment : positive
```

As you can see the model correctly predicts the sentiment for the randomly selected review from the test set. In the end, let's add the sentiment score (basically the probability) and the predicted sentiment to the data frame.

```
def add_score_predictions(data, reviews_list_idx):
    data['sentiment score'] = 0
    reviews_list_idx = pad_sequences(reviews_list_idx, maxlen=maxLen, padding='post')
    review_preds = model.predict(reviews_list_idx)
    data['sentiment score'] = review_preds
    pred_sentiment = np.array(list(map(lambda x : 'positive' if x > 0.5 else 'negative',review_preds)))
    data['predicted sentiment'] = 0
    data['predicted sentiment'] = pred_sentiment
    return data
```

This function returns us a modified data frame which looks like,

RSCOE-IT

	review	sentiment	sentiment score	predicted sentiment
0	one of the other reviewers has mentioned that ...	positive	0.559877	positive
1	a wonderful little production. the...	positive	0.987989	positive
2	i thought this was a wonderful way to spend ti...	positive	0.988815	positive
3	basically there's a family where a little boy ...	negative	0.053478	negative
4	petter mattei's "love in the time of money" is...	positive	0.990116	positive
...	...	...	...	...
49995	i thought this movie did a down right good job...	positive	0.889501	positive
49996	bad plot, bad dialogue, bad acting, idiotic di...	negative	0.020020	negative
49997	i am a catholic taught in parochial elementary...	negative	0.296422	negative
49998	i'm going to have to disagree with the previou...	negative	0.707072	positive
49999	no one expects the star trek movies to be high...	negative	0.064294	negative

50000 rows × 4 columns

Conclusion:

Thus We have Studied how to preprocess the given text and implement a binary classifier of movie review text using LSTMs.

RSCOE-IT